NEURAL NETWORKS / DAY 1

Neural network foundations

From tensors and single neurons to a network you can train and check by hand

Full day · 09:00–17:00 | 7 teaching parts | 2 laboratories | NumPy · scikit-learn · PyTorch

Today's plan

09:00	Welcome, five-day route, neural networks in context	Lecture
09:45	Tensors, shapes, and tools	Lecture
10:20	Break	Break
10:35	Lab 1: Sonar baseline pipeline, then debrief	Lab
11:40	From neuron to perceptron and XOR	Lecture
12:30	Lunch	Lunch
13:30	Multilayer networks; losses and gradient descent	Lecture
14:45	Break	Break
15:00	Backpropagation, training loop, first networks in code	Lecture
15:55	Lab 2: NumPy network on Sonar, debrief, exit check	Lab

Five days from a single neuron to attention

DAY 1 · TODAY

Neural network foundations

- Tensors and shapes
- Neuron, perceptron
- MLPs, activations
- Losses, gradients
- Backpropagation

Check a forward and backward pass

→

DAY 2

Training neural networks reliably

- Learning curves
- Initialization
- Normalization
- Optimizers
- Regularization

Reproduce a training experiment

→

DAY 3

Energy modelling capstone

- Data audits
- ESP gas lock
- PE log curve
- Sensor signals
- Error analysis

Defend a model comparison

→

DAY 4

Convolutional neural networks

- Convolution, pooling
- Architectures
- Transfer learning
- Segmentation
- Seismic, borehole

Check an image or signal model

→

DAY 5

Sequence models and attention

- Windows, baselines
- RNN, LSTM, TCN
- Attention
- Transformers
- Forecast intervals

Evaluate a future forecast

Schematic course map. Day 3 applies dense networks to the course energy data before CNNs and sequence models.

LEARNING OUTCOMES

What you will be able to do by 17:00

1. Place neural networks within AI and machine learning, and judge when one is worth trying.
2. Track batch, feature, and output shapes through a dense network.
3. Explain why XOR needs a hidden layer and what hidden units compute.
4. Match output layers, targets, and losses for regression and classification.
5. Compute one forward and backward pass by hand and verify it numerically.
6. Train a first network in NumPy, scikit-learn, or PyTorch against a baseline.

PRACTICE

4 min · Pairs

Warm-up: where would a network earn its place?

1. Name one prediction task from your work with a fixed set of numeric inputs.
2. What simple model would you compare a network against?
3. What evidence would convince your team that the network is better?

PART 1 OF 7

01

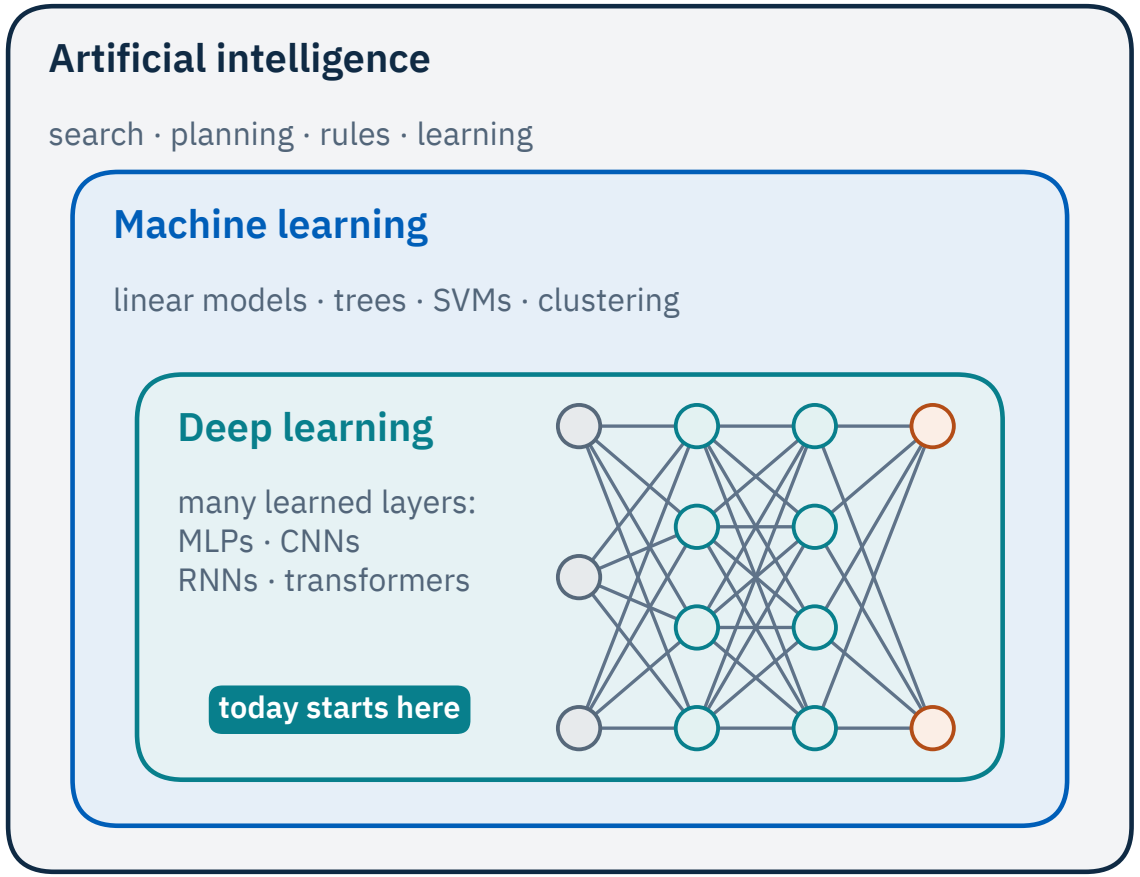
Neural networks in context

One model family among many; it must earn its place.

- AI, machine learning, and deep learning
- Eight milestones since 1943
- Where networks meet energy data
- When a network is worth trying

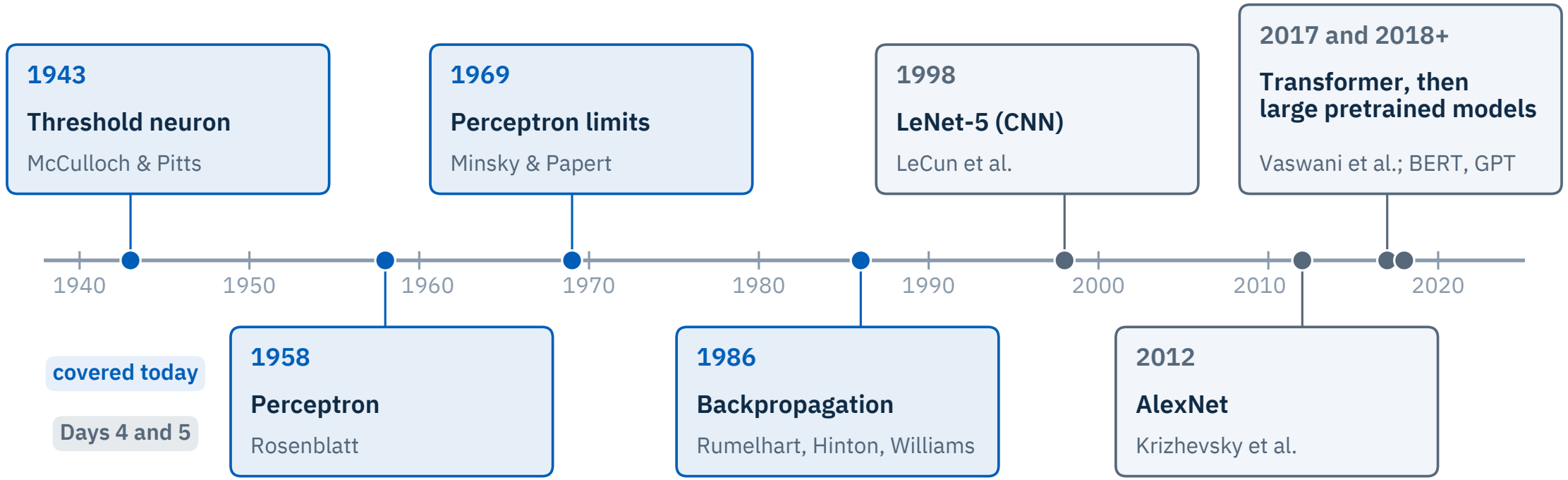
Deep learning is one branch of machine learning

- AI is the broad field; not all of it learns from data.
- Machine learning fits a model's parameters to data.
- **Deep learning** composes many learned layers.
- The same networks serve classification, regression, and representation learning.



Schematic. The examples in each field are illustrative, not exhaustive.

Eight milestones from threshold units to transformers



Schematic timeline; dates are publication years of the cited works. Blue: covered today; grey: covered on Days 4 and 5.

Data, compute, and software made depth practical

Data

- Large labelled collections, such as ImageNet
- Sensors and logs recorded at scale
- Energy data: often few independent units

Compute

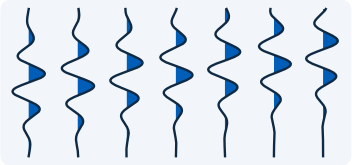
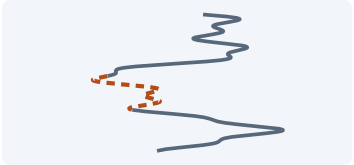
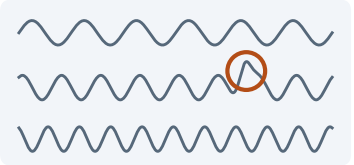
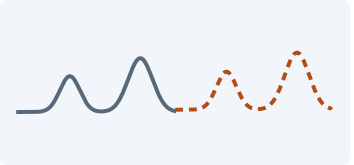
- Matrix products map onto GPU hardware
- Many training runs become affordable
- Larger and deeper models fit in memory

Methods and software

- ReLU units, better initialization, dropout
- Automatic differentiation frameworks
- Open-source code and pretrained models

◆ None of these removes the need for a baseline and an honest split.

Where neural networks meet energy data

Seismic interpretation	Log prediction	Production forecasting	Equipment health	Power and load forecasting
				
image patches (B, 1, H, W)	log curves (B, D)	rate history (B, T, F)	sensor channels (B, T, F)	weather, history (B, T, F)
CNN	MLP, 1-D CNN	LSTM, TCN	MLP, autoencoder	MLP, LSTM
fault or facies map	missing curve (PE)	future rates	anomaly score	next-day profile
Day 4	Day 3	Day 5	Days 2 to 5	Day 5

Schematic. Common data–architecture pairings, not recommendations; every case still needs a baseline. Chips show where the course covers each.

A neural network needs the same evidence as other models

1. **Frame** — the target, the inputs known at prediction time, the decision it supports.
2. **Baseline** — the simplest defensible non-neural model on the same rows.
3. **Split** — by well, asset, or time, matching how the model will be used.
4. **Select** — architecture, learning rate, and threshold on validation data only.
5. **Test** — once, on held-out data; then inspect errors by group.

◆ A flexible model exploits leakage as readily as any other model; it does not remove it.

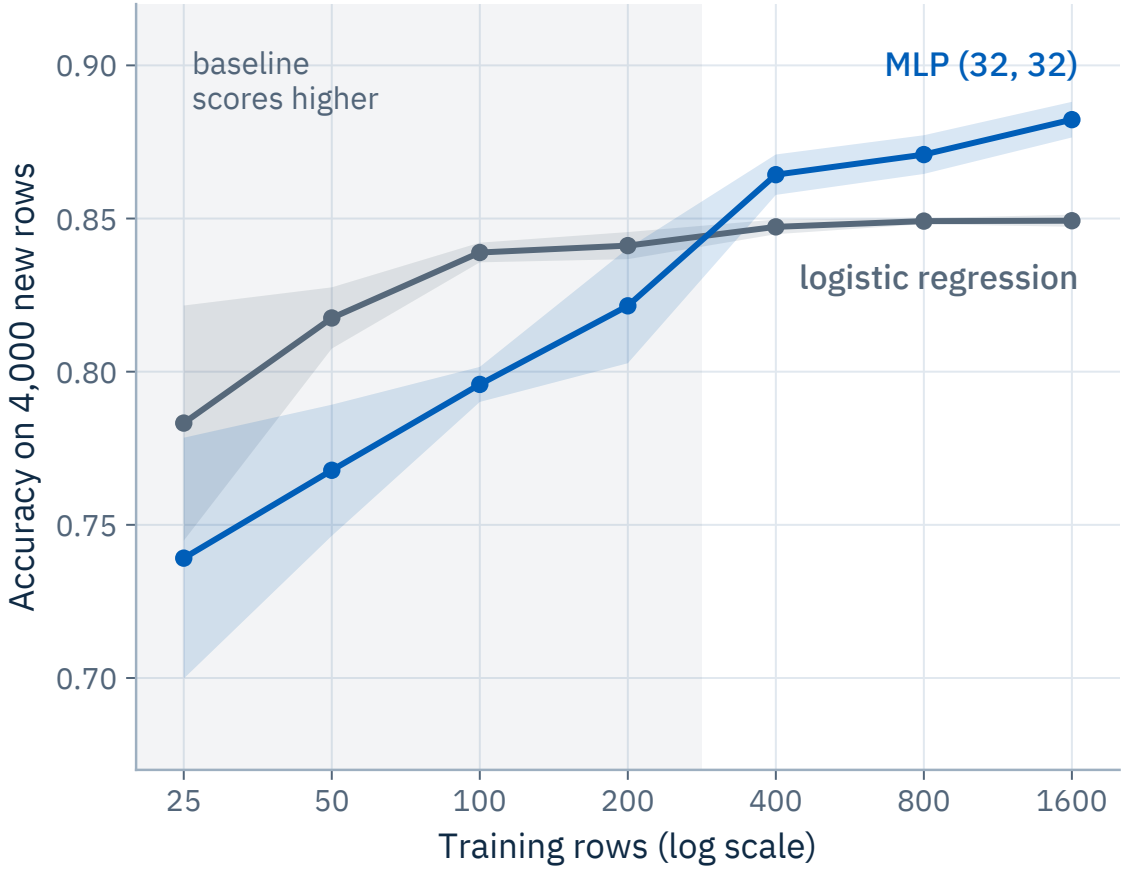
The data type decides whether a network is worth trying

Data and task	Useful initial comparison	Main concern
Small tabular dataset	Linear model and boosted trees	High variance from limited data
Images	CNN or pretrained visual model	Labels and domain shift
Ordered signals	Linear lag model, 1-D CNN, RNN	Correct sequence construction
Complex representations	Task-specific neural architecture	Compute and validation evidence

◆ Today's Sonar data (208 rows, 60 numeric features) is a small tabular problem: expect high variance.

On small samples, a simple baseline can beat a network

- Same synthetic task; only the number of training rows changes.
- Up to 200 rows, logistic regression scores higher.
- From 400 rows the MLP pulls ahead: 0.88 vs 0.85 at 1,600.
- In energy data, count independent wells or assets, not rows.



Computed: synthetic two moons plus 4 noise features; lines show the mean of 8 resamples (seeds 0–7), bands ±1 sd.

Flexibility is paid for in data, compute, and care

✓ What networks offer

- Learn features from raw signals and images
- One framework for many output types
- Transfer from pretrained models

vs

✗ What they cost

- More data, tuning, and compute
- More ways to fail silently
- Harder to explain and to validate

◆ Much of today builds habits that turn silent failures into loud ones.

PRACTICE

5 min · Pairs

Would you start with a neural network here?

1. 300 wells, 12 log-derived features per depth sample → facies class.
2. Two years of 1-minute ESP sensor data → gas-lock warning an hour ahead.
3. 50,000 labelled seismic image patches → fault or no fault.
4. Daily wind-farm output and weather forecasts → next-day power.

PART 2 OF 7

02

Tensors, shapes, and tools

Most first bugs show in the shapes before the loss.

- The Python stack and what each tool is for
- Tensors, ranks, and the batch axis
- Broadcasting, vectorization, and devices
- The Sonar data and a baseline pipeline

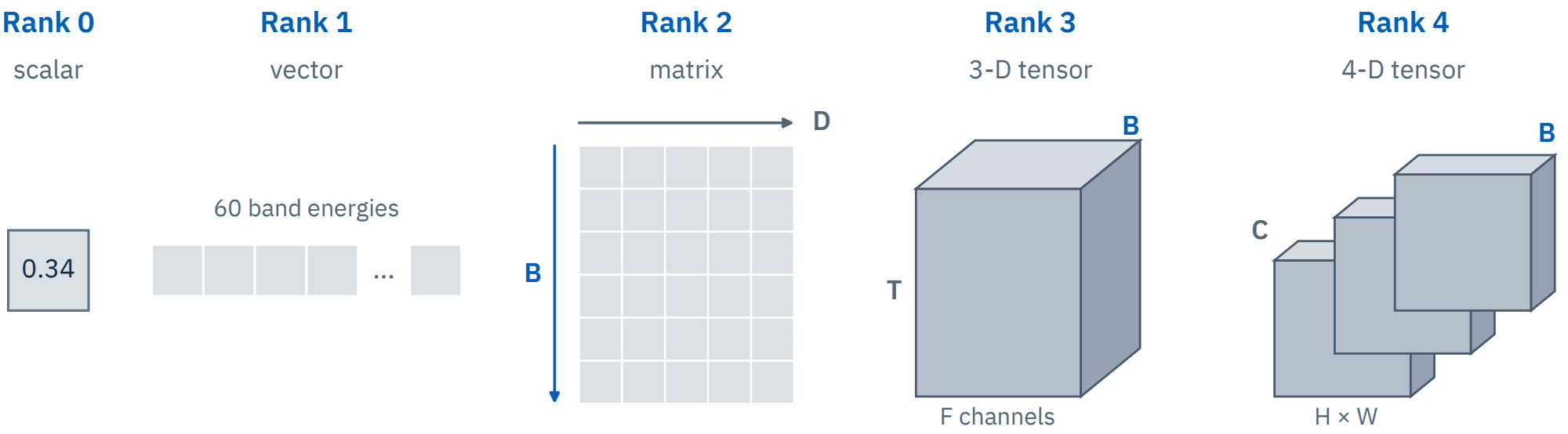
The Python stack has distinct responsibilities

Tool	Responsibility	First check
NumPy	Arrays and vectorized calculations	Shape and dtype
pandas	Tables, labels, joins, and timestamps	Row alignment and schema
Matplotlib	Scientific plots	Axes, units, and sampling
scikit-learn	Baselines and preprocessing	Fitting inside training folds
PyTorch / Keras	Differentiable models and optimization	Output-loss compatibility

◆ Keep notebooks restartable: run top to bottom before trusting any number.

A tensor is an array with a fixed number of axes

The **rank** is the number of axes; the shape lists the length of each axis.



Schematic. B = batch, D = features, T = time steps, F = channels, C = image channels, H × W = image size.

Energy data arrive in a few standard tensor shapes

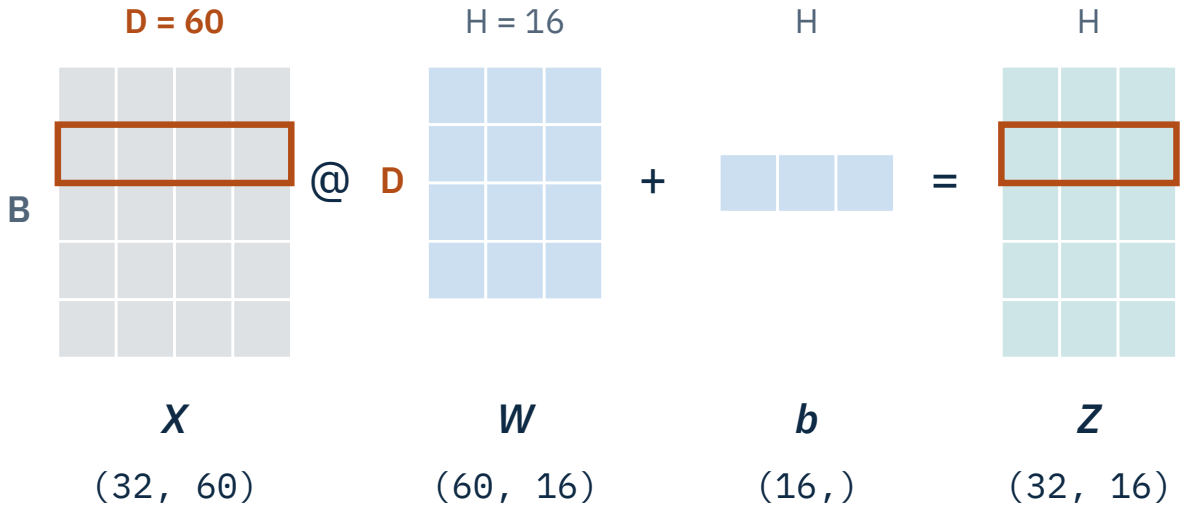
Data	Tensor shape	Axes
Log samples as a table	(B, D)	depth samples × curves
Sonar echoes (today)	(B, 60)	echoes × frequency bands
ESP sensor windows	(B, T, F)	windows × time steps × channels
Seismic patches	(B, 1, H, W)	patches × amplitude channel × height × width
Load or production history	(B, T, 1)	windows × time steps × one value

◆ First question for any dataset: what is one example, and what does B count?

One batch, one shape contract

$$Z = XW + b$$

- Rows are examples: X is (B, D), Z is (B, H).
- Inner sizes must match: (B, D) @ (D, H).
- W never contains B, so the batch size is free.
- PyTorch `nn.Linear(D, H)` stores its weight as (H, D), the transpose of this W.



`(32, 60) @ (60, 16)` inner sizes must match

row i of Z uses only row i of X (outlined)

Schematic with B = 32, D = 60, H = 16. Row i of Z depends only on row i of X.

Broadcasting stretches size-1 axes, sometimes silently

- Shapes align from the last axis; missing or size-1 axes are copied.
- A bias of shape (H,) added to (B, H) is the intended use.
- (B, 1) minus (B,) silently becomes (B, B).
- Reshape targets to match predictions before any loss.

Adding a bias: (3, 2) + (2,) ✓

1	2		10	20		11	22
3	4	+			=	13	24
5	6					15	26

b is copied down the batch

A loss with mismatched shapes: (3, 1) − (3,) ✗

0.9			1	0	1	−0.1	0.9	−0.1
0.2	−				=	−0.8	0.2	−0.8
0.7						−0.3	0.7	−0.3

result: (3, 3)

diagonal = intended

others = nonsense

MSE intended 0.047 · MSE computed 0.313 · no error raised

Exact NumPy arithmetic. The mismatched loss is 0.313 instead of 0.047, and no error is raised.

PRACTICE

4 min · Individual

Predict the shape before you run the cell

X has shape (32, 60), W (60, 16), b (16,), y_pred (32, 1), and y (32,).

1. `X @ W + b`
2. `(X @ W).mean(axis=0)`
3. `y_pred - y`
4. `W @ X`

Vectorized calculation replaces nested loops

```python
import numpy as np

X = np.array([[1., 2.], [3., 4.]])      # 2 examples, 2 features
W = np.array([[0.5, -1.], [1., 0.5]])   # 2 inputs -> 2 units
b = np.array([0., 1.])
Z = X @ W + b                           # (2, 2): one row per example
A = np.maximum(Z, 0.)                   # ReLU, elementwise
assert A.shape == (2, 2)
print(A)
```

OUTPUT

```
[[2.5 1. ]
 [5.5 0. ]]
```

One matrix product computes every unit for every example at once.

PyTorch tensors mirror NumPy arrays

```
import numpy as np
import torch

a = np.arange(6, dtype=np.float32).reshape(2, 3)
t = torch.from_numpy(a)              # shares memory with a
print(t.shape, t.dtype, t.ndim)
print((t @ t.T).shape)               # (2, 3) @ (3, 2)
a[0, 0] = 100.                       # change the NumPy array...
print(t[0, 0])                       # ...and the tensor sees it
```

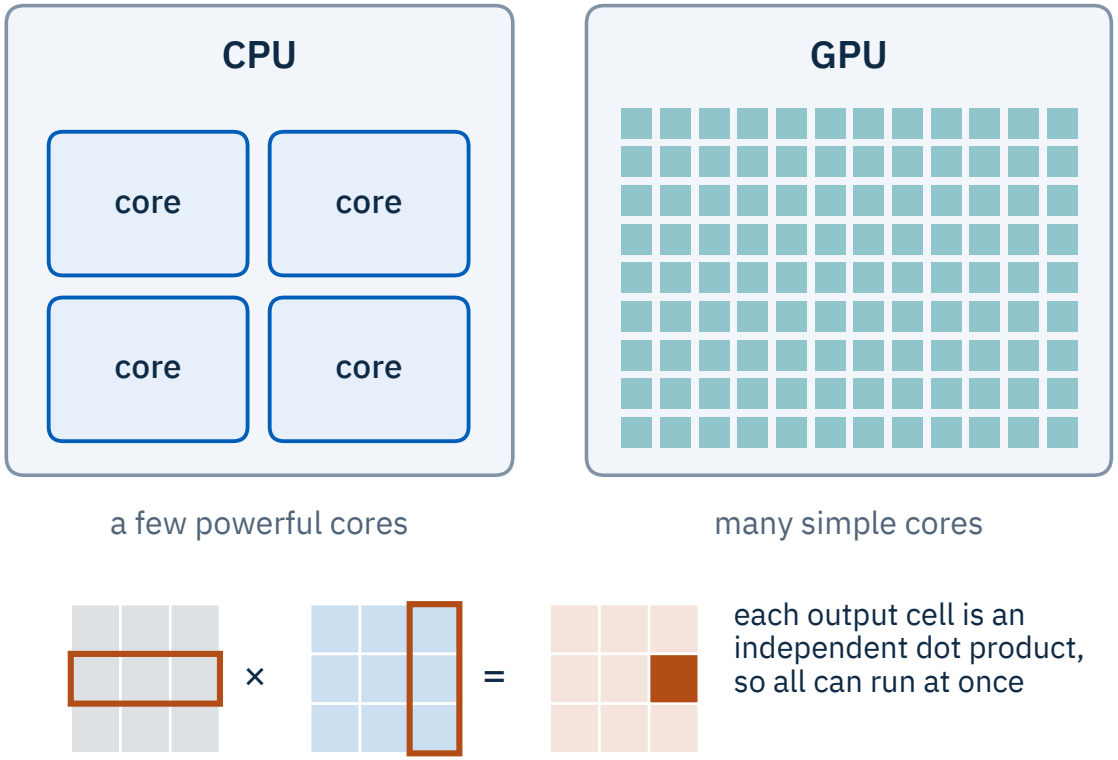
OUTPUT

```
torch.Size([2, 3]) torch.float32 2
torch.Size([2, 2])
tensor(100.)
```

`torch.from_numpy` shares memory; `torch.tensor` copies. NumPy defaults to float64, PyTorch layers to float32.

GPUs run many identical operations at once

- A matrix product is many independent dot products.
- GPUs trade per-core flexibility for massive parallelism.
- Small tabular networks, like today's, train quickly on a CPU.
- Model and data must be on the same device.



Schematic; core counts are illustrative, not a specific device.

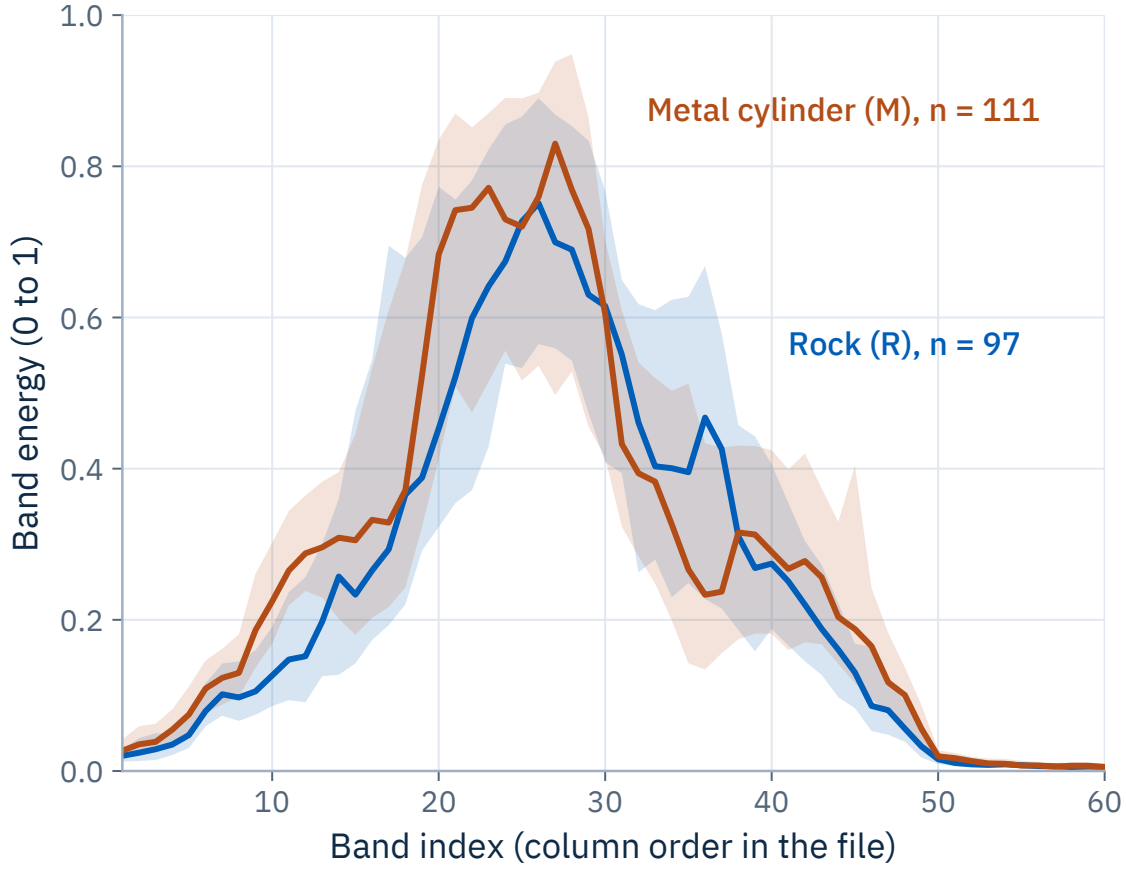
A reliable table-to-tensor path

1. **Join** — attach labels by a verified key, never by row position after a sort.
2. **Select** — list predictor columns explicitly and keep their order fixed.
3. **Split** — by well, asset, or time before fitting anything.
4. **Scale** — fit imputers and scalers on training rows only.
5. **Convert** — cast to float32 and check for NaN and infinite values.

- Numeric row IDs are not features: drop them before training.

Lab data: 208 sonar echoes with 60 band energies each

- Labels: R = rock, M = metal cylinder, not "mineral".
- 97 rock and 111 metal echoes; values already in [0, 1].
- The classes overlap heavily in every single band.
- With 208 rows, one random split gives a noisy score.



Computed from sonar.all-data.csv: median and interquartile range of each band for each class.

Baseline pipeline for Sonar

```
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import StratifiedKFold, cross_val_score
from sklearn.model_selection import train_test_split
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y_raw, random_state=42)
baseline = make_pipeline(StandardScaler(),
                         LogisticRegression(max_iter=1000))
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=0)
scores = cross_val_score(baseline, X_train, y_train, cv=cv)
print(f"CV accuracy {scores.mean():.3f} +/- {scores.std():.3f}")
```

OUTPUT

```
CV accuracy 0.753 +/- 0.061
```

The 42 test rows stay untouched until one final evaluation.

LABORATORY

50 min

Lab 1: a Sonar baseline you can defend

- Load, explore, and encode the labels; print the label mapping.
- Build three pipelines: logistic regression, random forest, RBF SVM.
- Compare them with shuffled 5-fold CV on the training rows only.
- Tune the best of them with GridSearchCV on the same folds.
- Test the tuned model once; note what 42 rows can establish.

NOTEBOOK

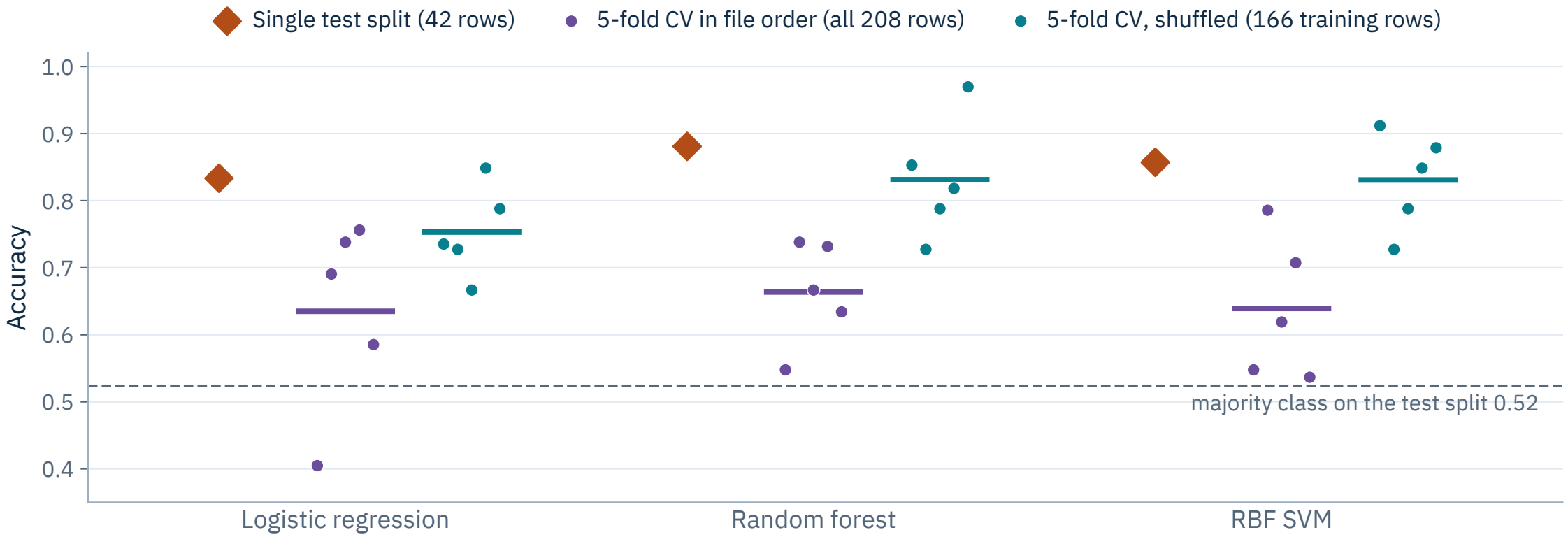
`exercises/01_sklearn_pipeline/sklearn_Pipeline_Student.ipynb`

EVIDENCE TO SUBMIT

- Score table: CV mean ± sd per model, one final test score
- The printed label mapping and class counts
- Two sentences on the uncertainty of your best score

DISCUSSION

Lab 1 debrief: the evaluation protocol moves the score



Computed from sonar.all-data.csv with the Lab 1 pipelines. Dots: single folds; bars: fold means; diamonds: the 42-row test split.

DISCUSSION

Lab 1 debrief: three habits to keep all week

Print the label mapping

- Mine = 1, Rock = 0 in all labs
- LabelEncoder picks M = 0
- Precision and recall depend on it

Keep the test set for last

- Choosing by test accuracy spends the test set
- Select with CV, then test once
- Report the test score with its n

Treat row order as structure

- Unshuffled folds follow the file order
- The two protocols differ by over 10 points
- Group or order splits when metadata allows

◆ The baseline to beat this afternoon: 0.75 CV accuracy (logistic), 0.83 (forest, SVM).

PART 3 OF 7

03

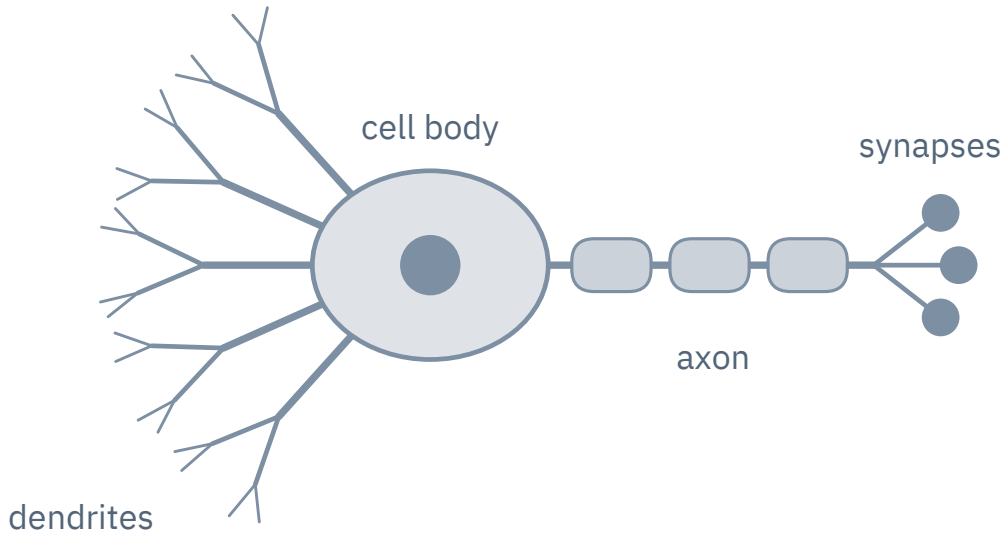
From neuron to perceptron

A weighted sum and a threshold draw one straight line.

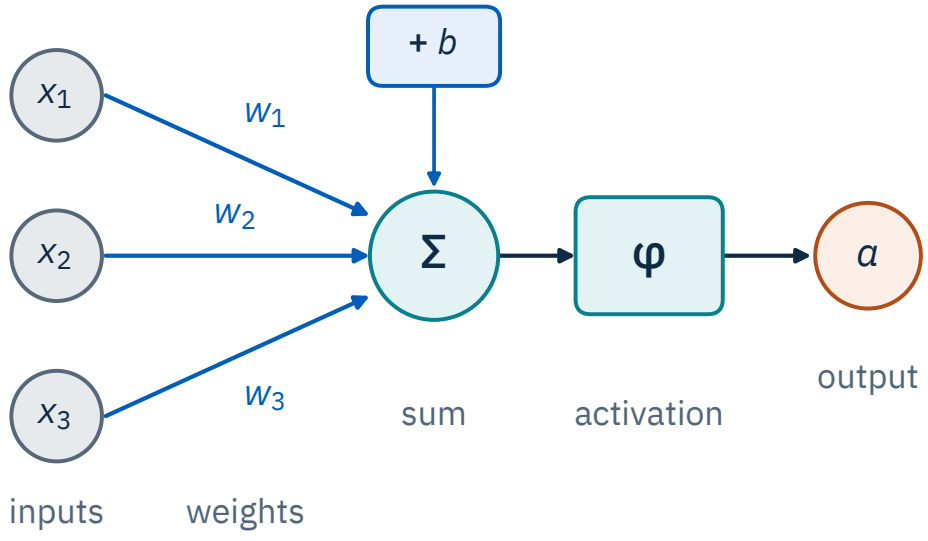
- Biological inspiration and its limits
- The artificial neuron and the perceptron rule
- Linear separability and XOR
- A hidden layer that solves XOR

The artificial neuron borrows a loose biological metaphor

Biological neuron (sketch)



Artificial unit (a formula)



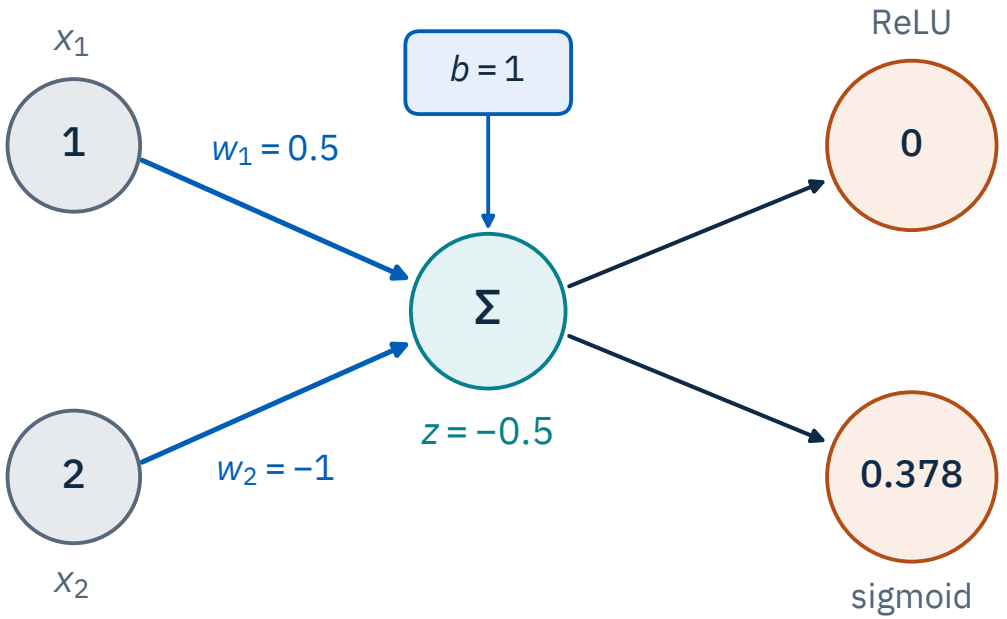
What the analogy leaves out: spike timing · computation inside dendrites · chemical signalling · adaptation over time

Schematic drawing, not anatomically accurate. The analogy motivated the model; the equations define it.

Weighted inputs, bias, and activation

$$z = \sum_{j=1}^{d} w_j x_j + b, \qquad a = \phi(z)$$

- The weights set each input's contribution.
- The bias shifts the operating point.
- The **activation** φ shapes the response.



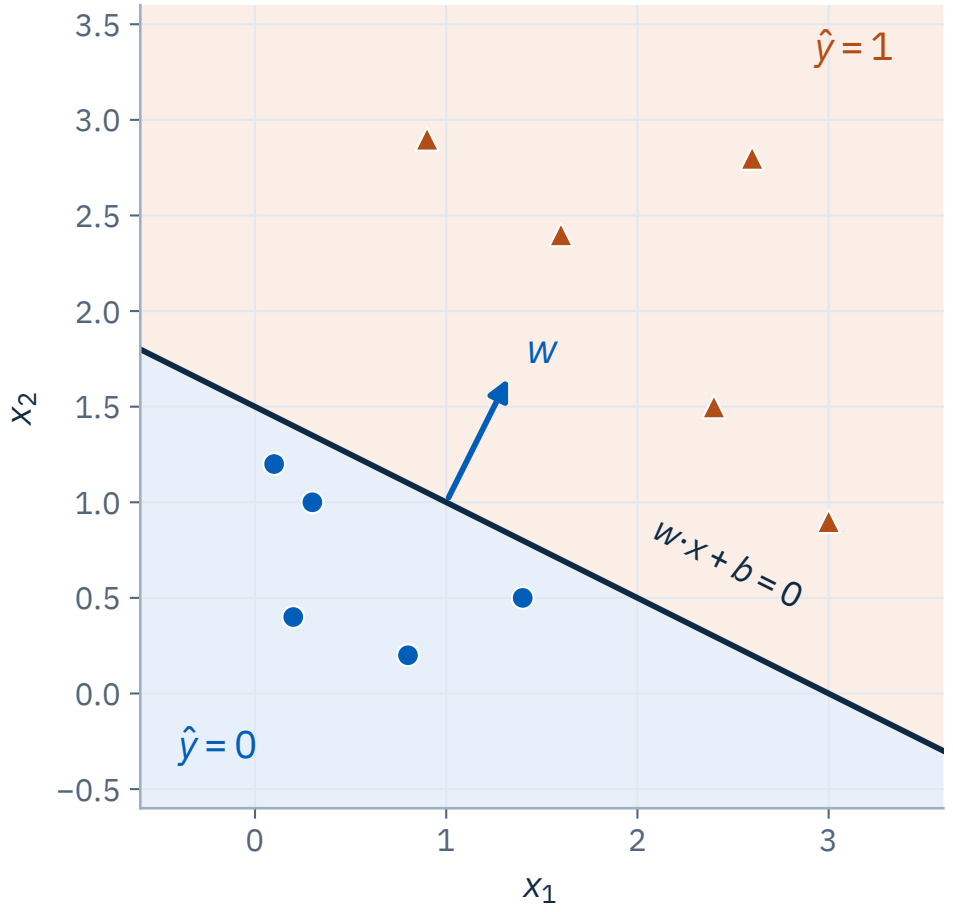
$$z = 0.5 \cdot 1 + (-1) \cdot 2 + 1 = -0.5$$

Exact arithmetic for inputs (1, 2), weights (0.5, −1), and bias 1.

A perceptron thresholds a weighted sum into a class

$$\hat{y} = \text{step}(w \cdot x + b)$$

- step(z) = 1 if z > 0, otherwise 0.
- The boundary $w \cdot x + b = 0$ is a straight line (a plane in more dimensions).
- The weight vector w is perpendicular to the boundary; b shifts it.

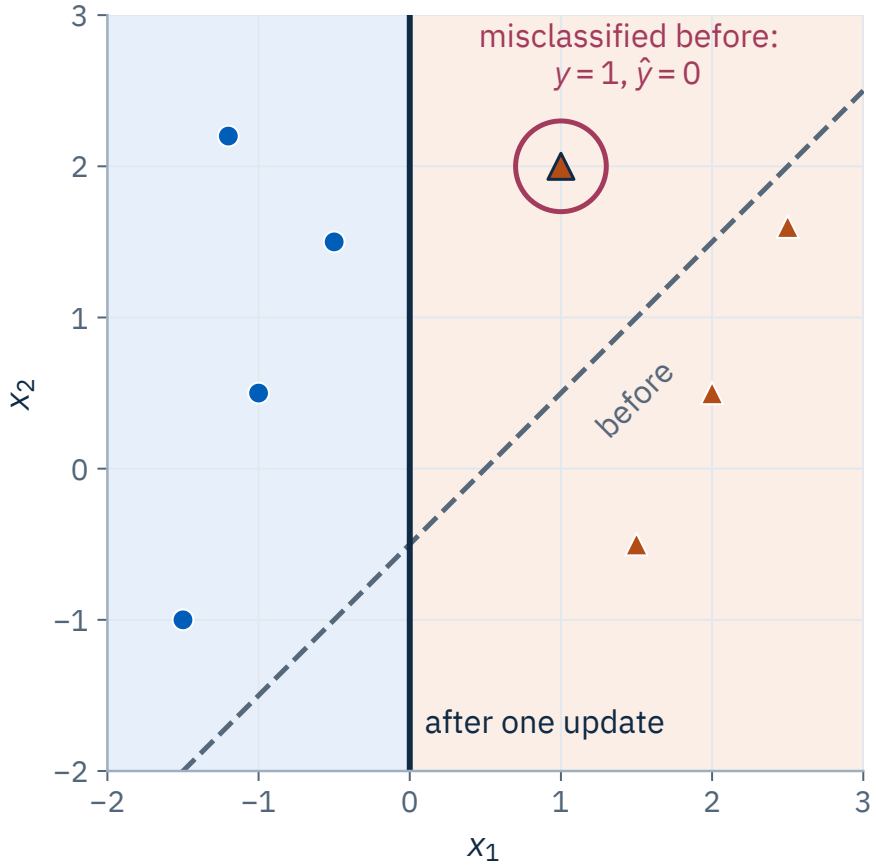


Exact geometry for w = (1, 2) and b = −3; illustrative points.

The perceptron rule moves the boundary toward mistakes

$$w \leftarrow w + \eta\,(y - \hat{y})\,x, \qquad b \leftarrow b + \eta\,(y - \hat{y})$$

- Correct predictions leave the weights unchanged.
- Worked update: $x = (1, 2)$, $y = 1$, $\hat{y} = 0$, $\eta = 0.5$.
- w: $(1, -1) \to (1.5, 0)$; b: $-0.5 \to 0$.
- On separable data the rule stops after finitely many mistakes.



Exact update. Dashed: boundary before; solid: after one update. The ringed point is now classified correctly.

PRACTICE

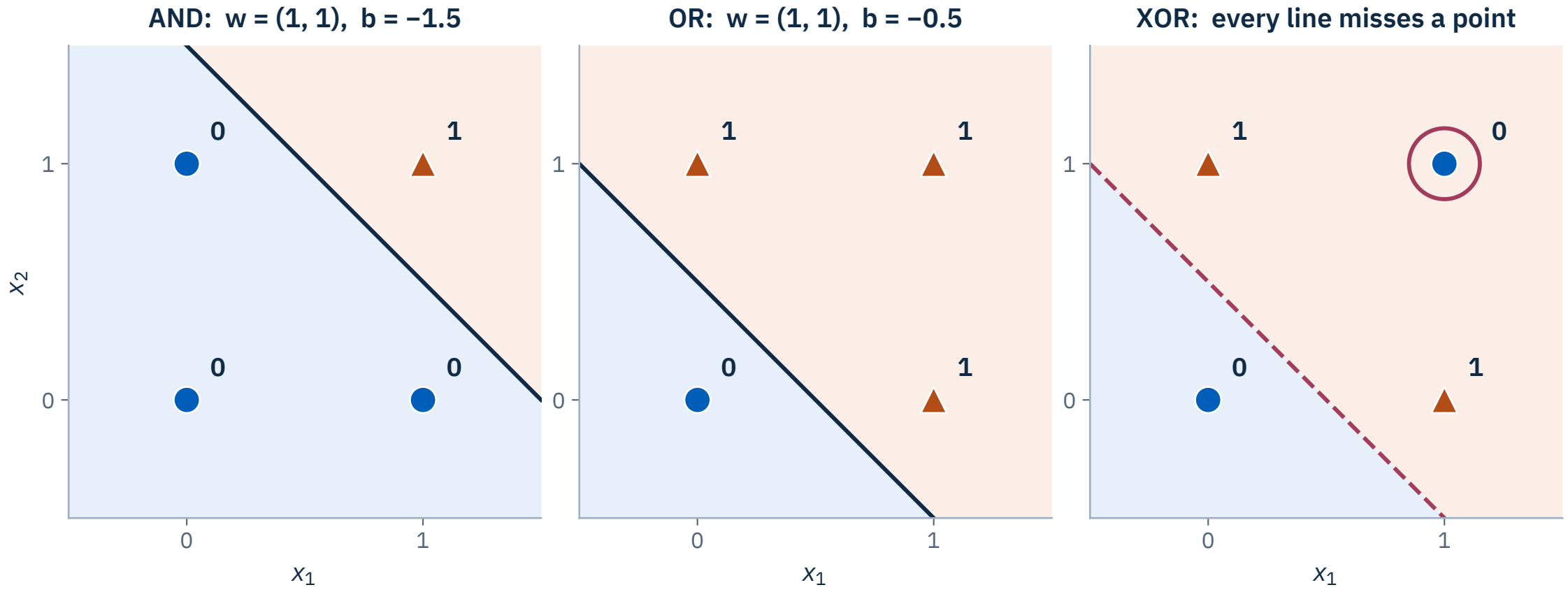
5 min · Individual

Update a perceptron by hand

Start from $w = (0.2, -0.4)$, $b = 0$, and learning rate $\eta = 0.5$.

1. Present $x = (1, 1)$ with $y = 1$: compute z, $\hat{y}$, and the new w and b.
2. Then present $x = (2, -1)$ with $y = 0$ and update again.
3. Are both examples now classified correctly?

AND and OR are linearly separable; XOR is not



Exact threshold units. On XOR even the best line, here the OR line, misclassifies (1, 1); the perceptron rule keeps making mistakes on every pass.

PRACTICE

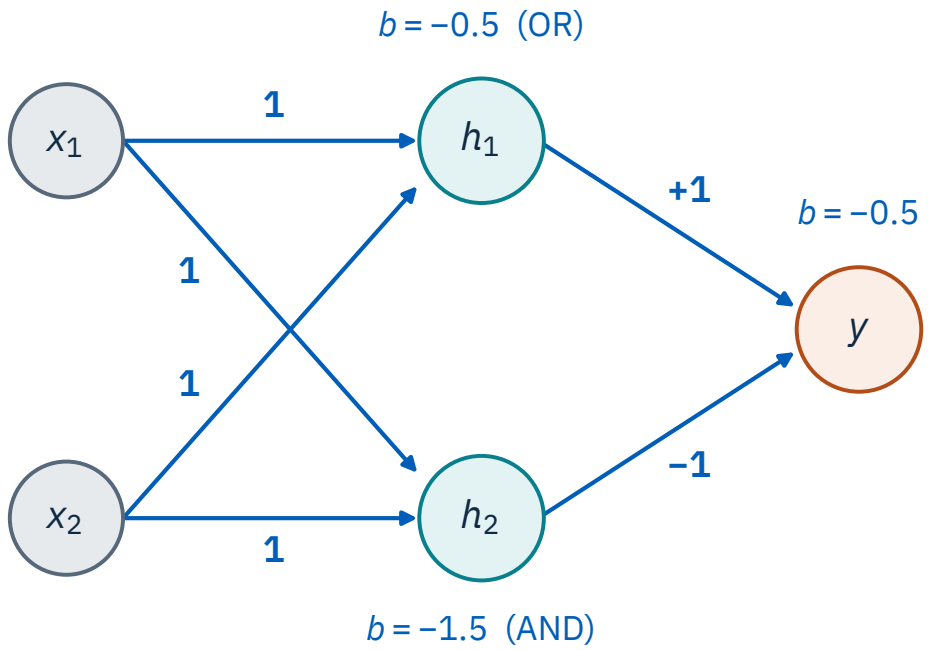
5 min · Pairs

Show that no single threshold unit computes XOR

A unit outputs 1 when $w_1x_1 + w_2x_2 + b > 0$ and 0 otherwise.

1. Write the four inequalities that XOR requires.
2. Add the two inequalities for the inputs (0, 1) and (1, 0).
3. Show that the result contradicts the other two.

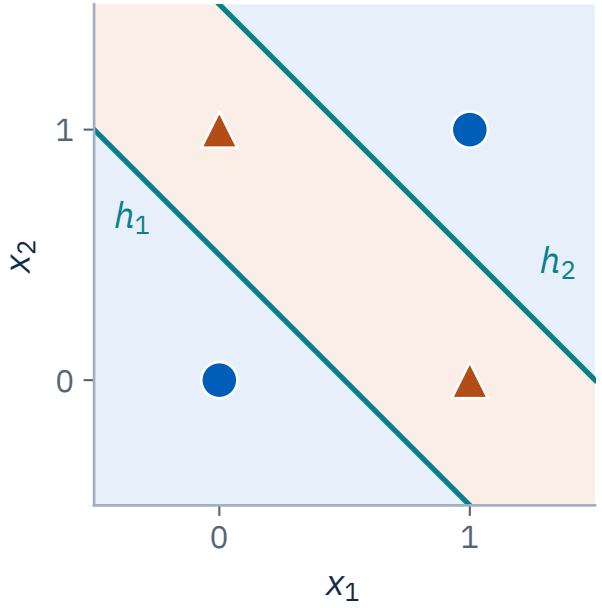
Two hidden units solve XOR: OR and not AND



all units: step(z) = 1 if z > 0, else 0

x_1	x_2	h_1	h_2	y
0	0	0	0	**0**
0	1	1	0	**1**
1	0	1	0	**1**
1	1	1	1	**0**

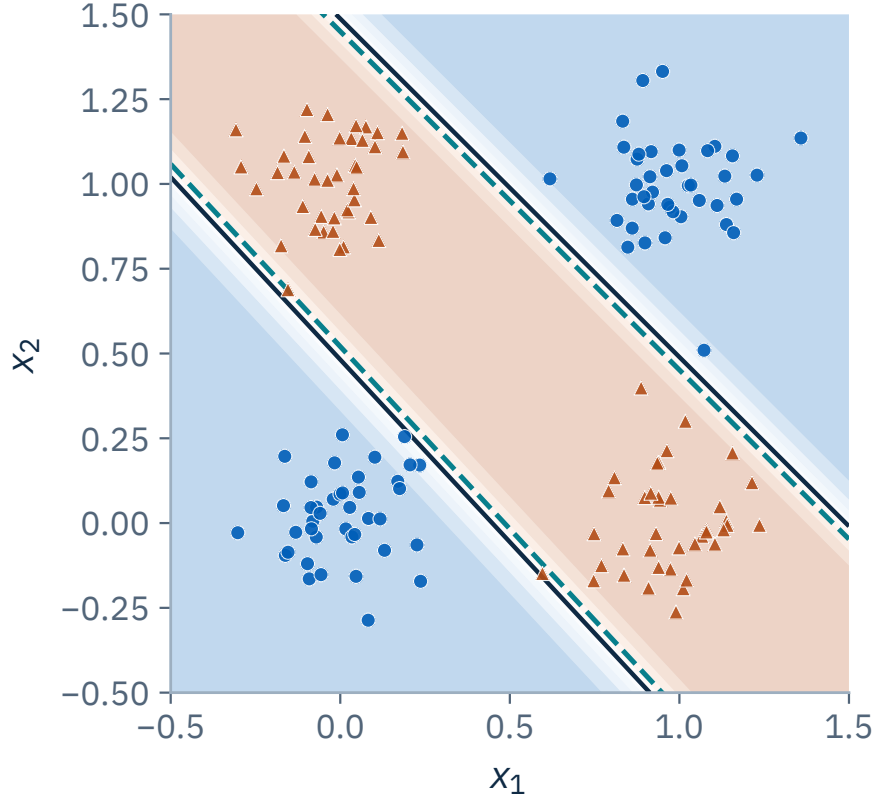
$y = h_1$ AND NOT h_2 = XOR



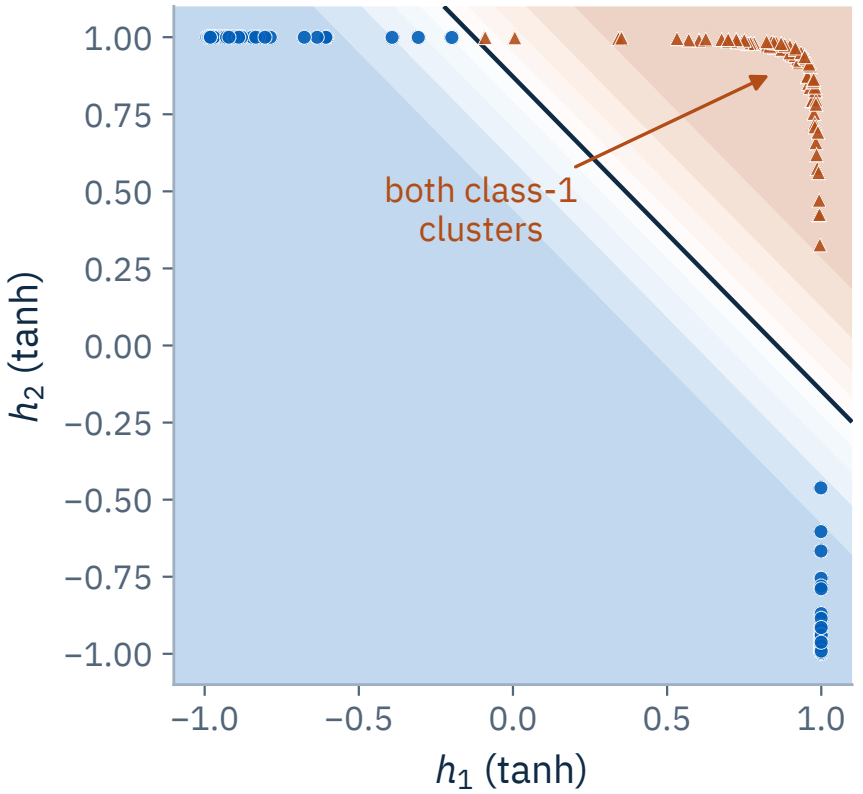
Exact hand-set weights with step activations. Hidden unit 1 is the OR line, hidden unit 2 the AND line; the output fires in the band between them.

The hidden layer re-maps inputs so that a line suffices

Input space: two hidden lines form a band



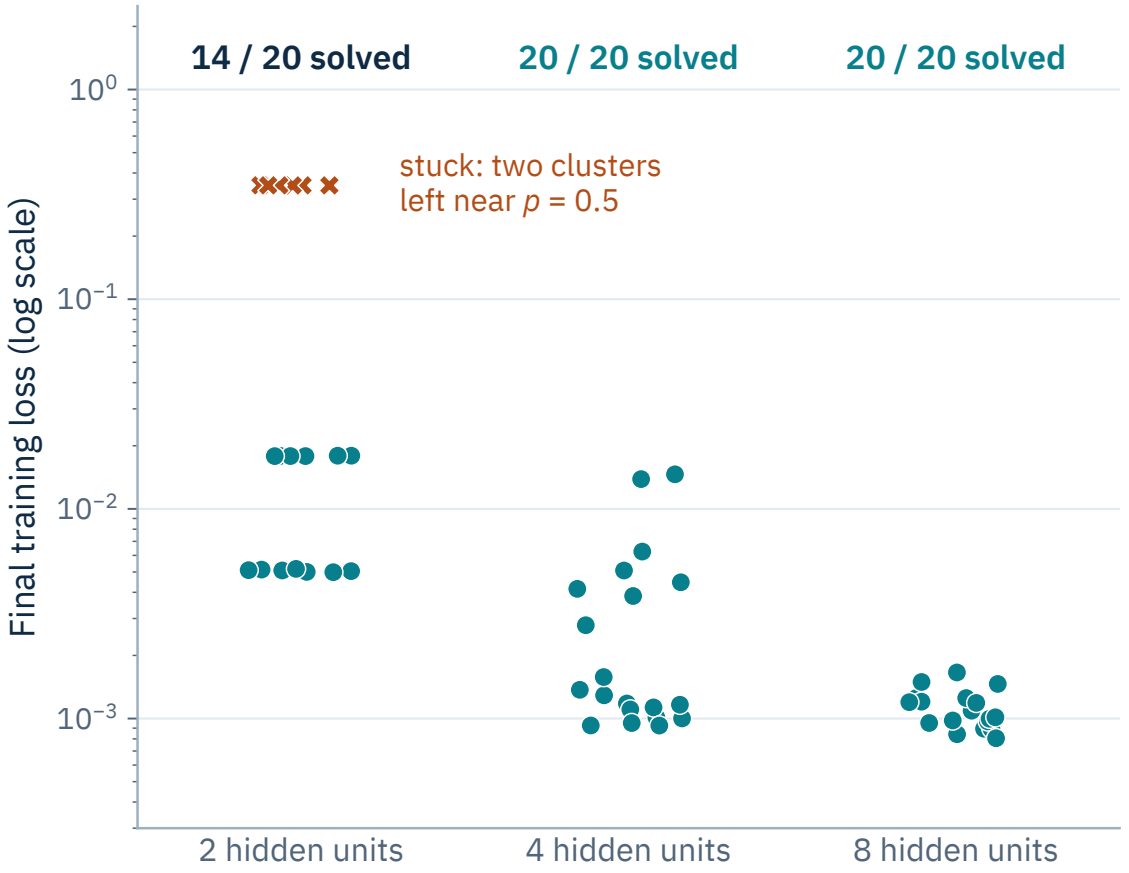
Hidden space: a straight line suffices



Computed: noisy XOR (4 × 40 points, seed 0); 2–2–1 tanh network, initialization seed 2, 300 full-batch gradient steps. Solid lines: decision boundary (p = 0.5); dashed: where each hidden unit is zero.

Gradient descent solves XOR, but not from every start

- Same data and training recipe; only the initial weights differ.
- 2 hidden units: 14 of 20 starts solve XOR.
- The others stall with two clusters left at $p \approx 0.5$.
- 4 or 8 hidden units: all 20 starts solve it.



Computed on the same noisy XOR data: 20 initialization seeds per width, 4000 full-batch steps, learning rate 1.0.

PRACTICE

3 min · Pairs

Which of these can a single perceptron represent?

Inputs are 0 or 1 unless stated; a unit fires when $w \cdot x + b > 0$.

1. NAND of two inputs
2. XNOR: 1 when both inputs are equal
3. Majority vote of three inputs
4. "$x_1 > x_2$" for real-valued inputs

PART 4 OF 7

04

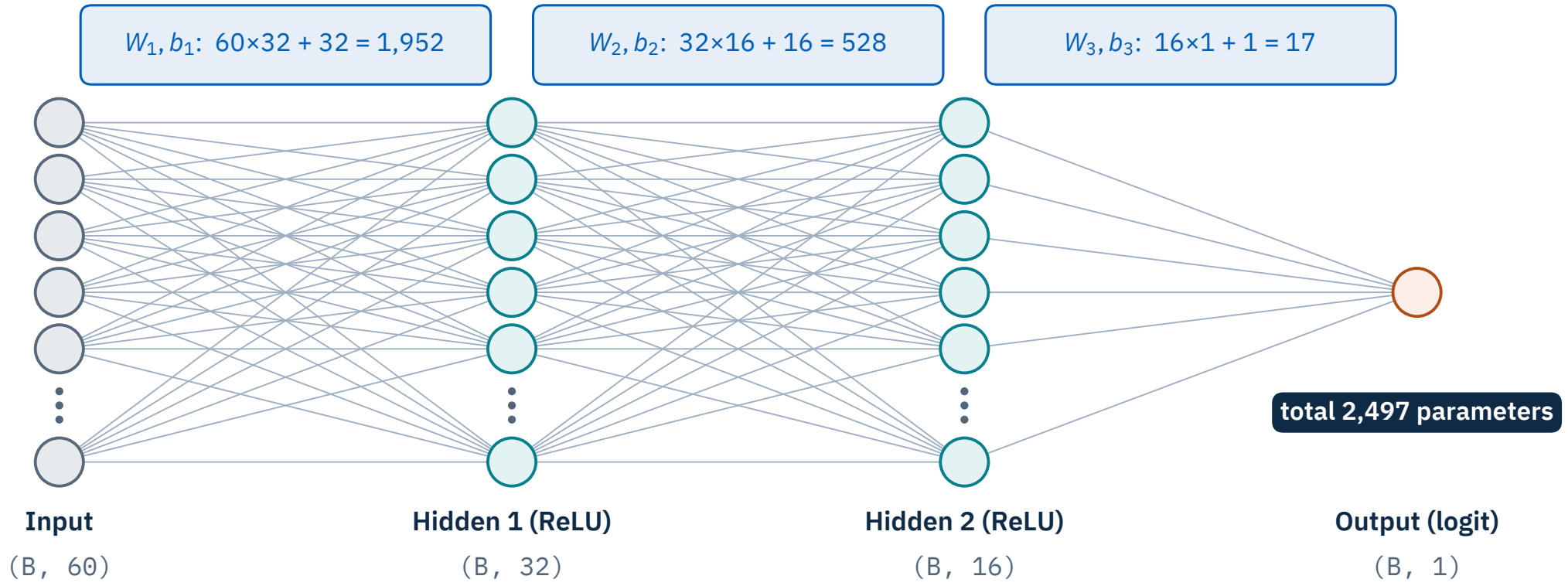
Multilayer networks

Stacked weighted sums and nonlinearities bend to the data.

- Layers as matrix products
- Why activations must be nonlinear
- Activation functions and their derivatives
- Universal approximation, width, and depth

A multilayer perceptron is a chain of dense layers

Each layer maps (B, in) to (B, out) with a weight matrix (in, out) and one bias per output unit.

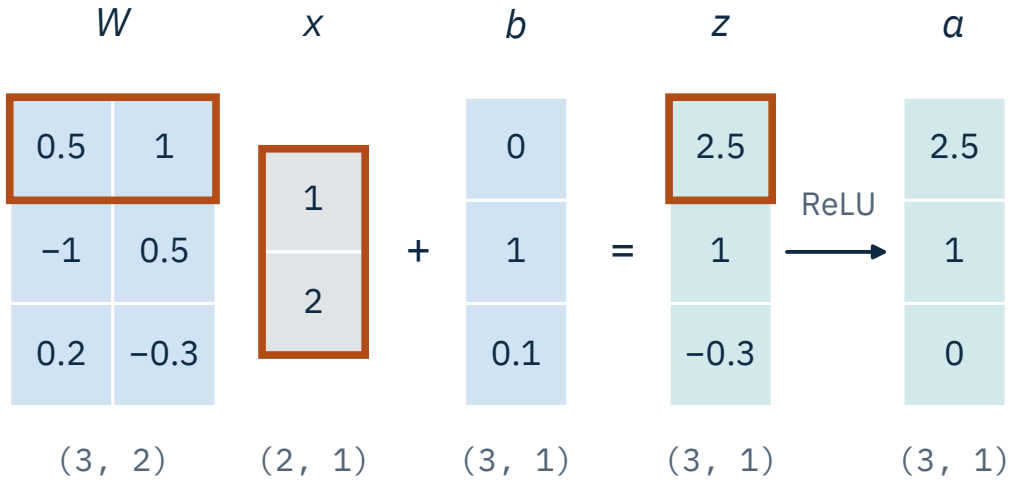


Schematic of a 60–32–16–1 network for Sonar-sized input; parameter counts are exact.

A dense layer is one matrix product plus a bias

$$z = Wx + b, \qquad a = \mathrm{ReLU}(z)$$

- One example in as a column vector *x*, one vector *a* out.
- Row *j* of *W* holds the weights of unit *j*.
- A batch stacks examples as rows: $Z = XW^{\top} + b$.



$z_1 = 0.5 \cdot 1 + 1.0 \cdot 2 + 0 = 2.5$

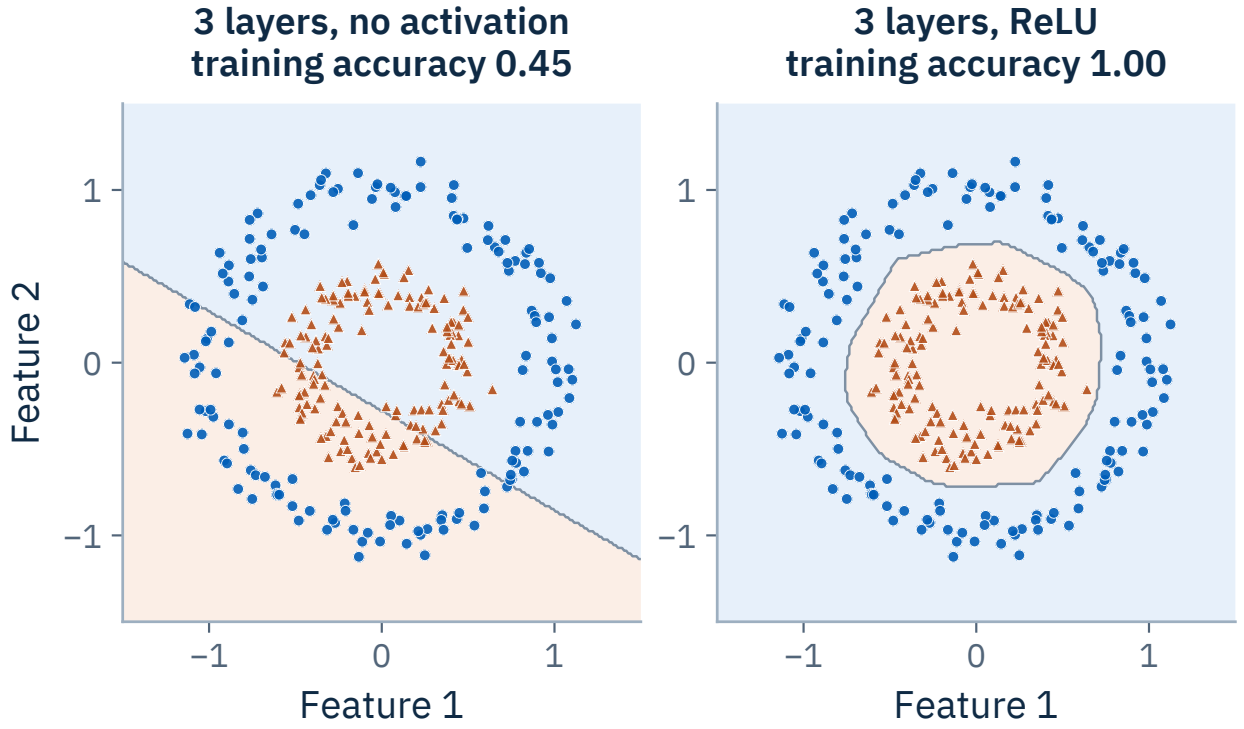
row *j* of *W* holds the weights of unit *j*

Exact arithmetic: z = (2.5, 1.0, −0.3), so a = (2.5, 1.0, 0).

Stacking linear layers is still linear

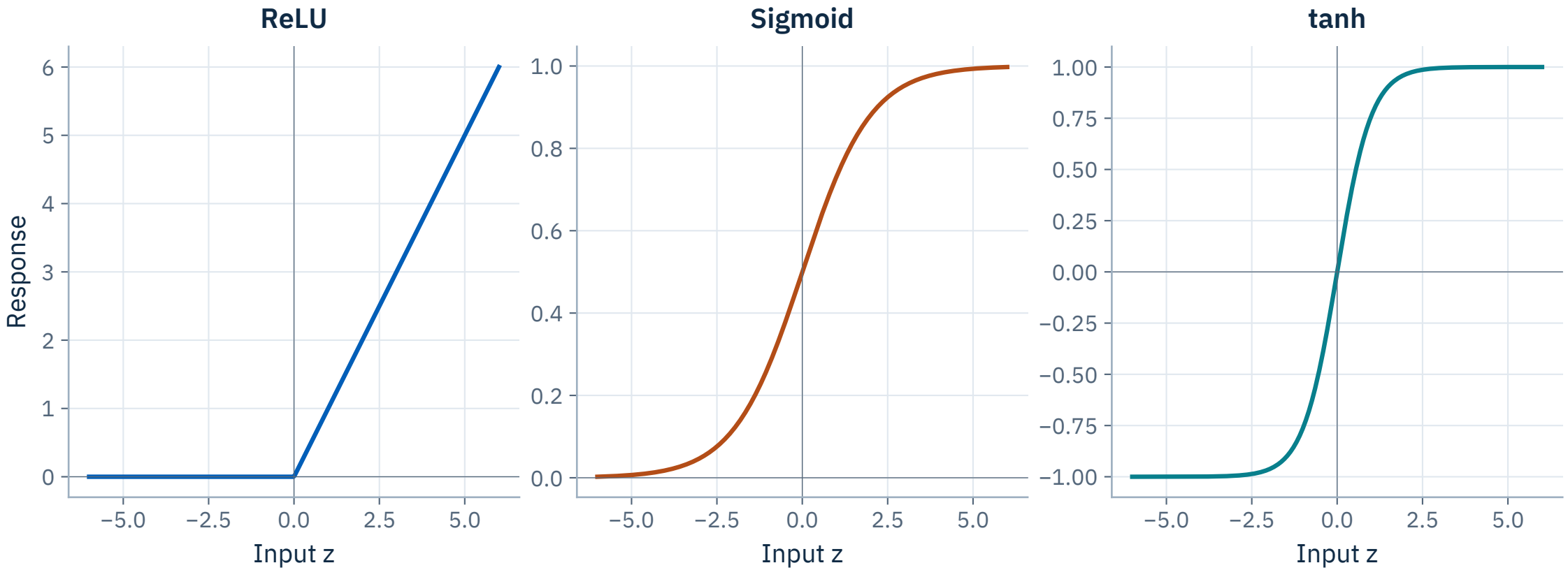
$$W_2(W_1x + b_1) + b_2 = (W_2W_1)x + (W_2b_1 + b_2)$$

- Without activations, any stack of layers collapses into one.
- Nonlinear activations let hidden units bend the boundary.
- Same data, same three layers: only the activation differs.



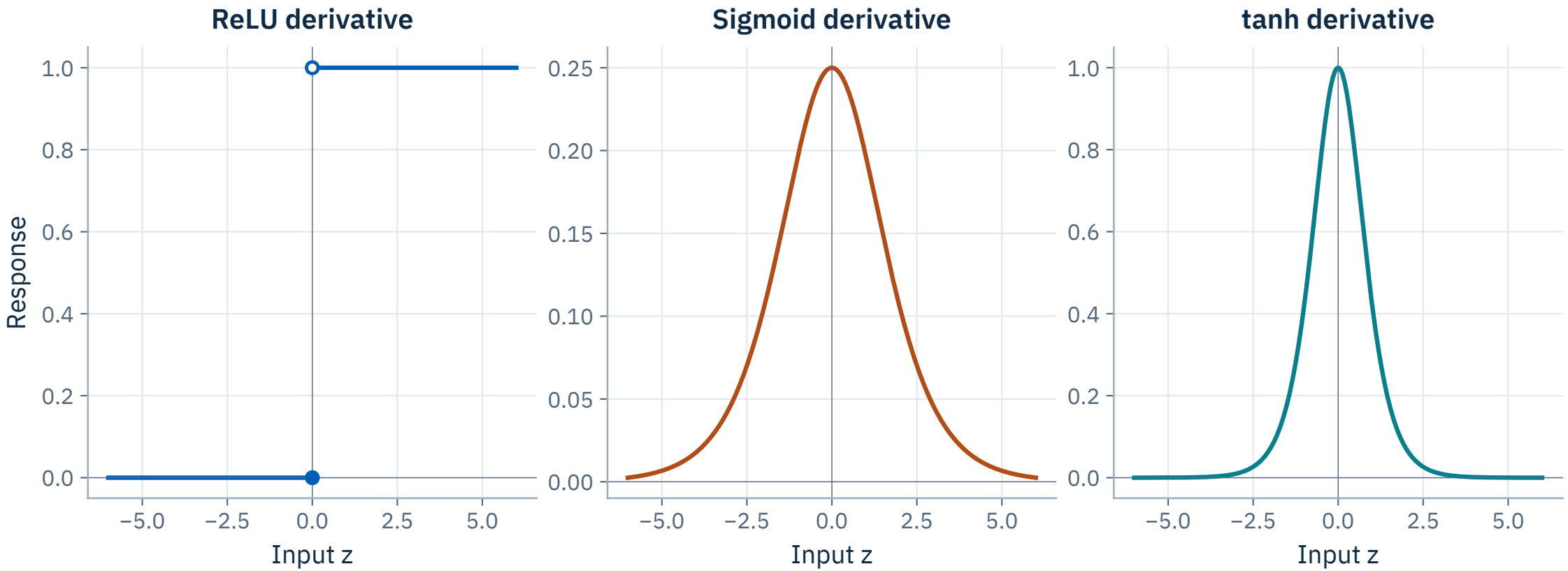
Computed: synthetic circles (seed 0); scikit-learn MLPClassifier with three hidden layers of 16 units.

Activation functions respond differently



Exact activation functions over a common input range. ReLU, sigmoid, and tanh have different ranges and saturation behaviour.

Derivatives explain saturation

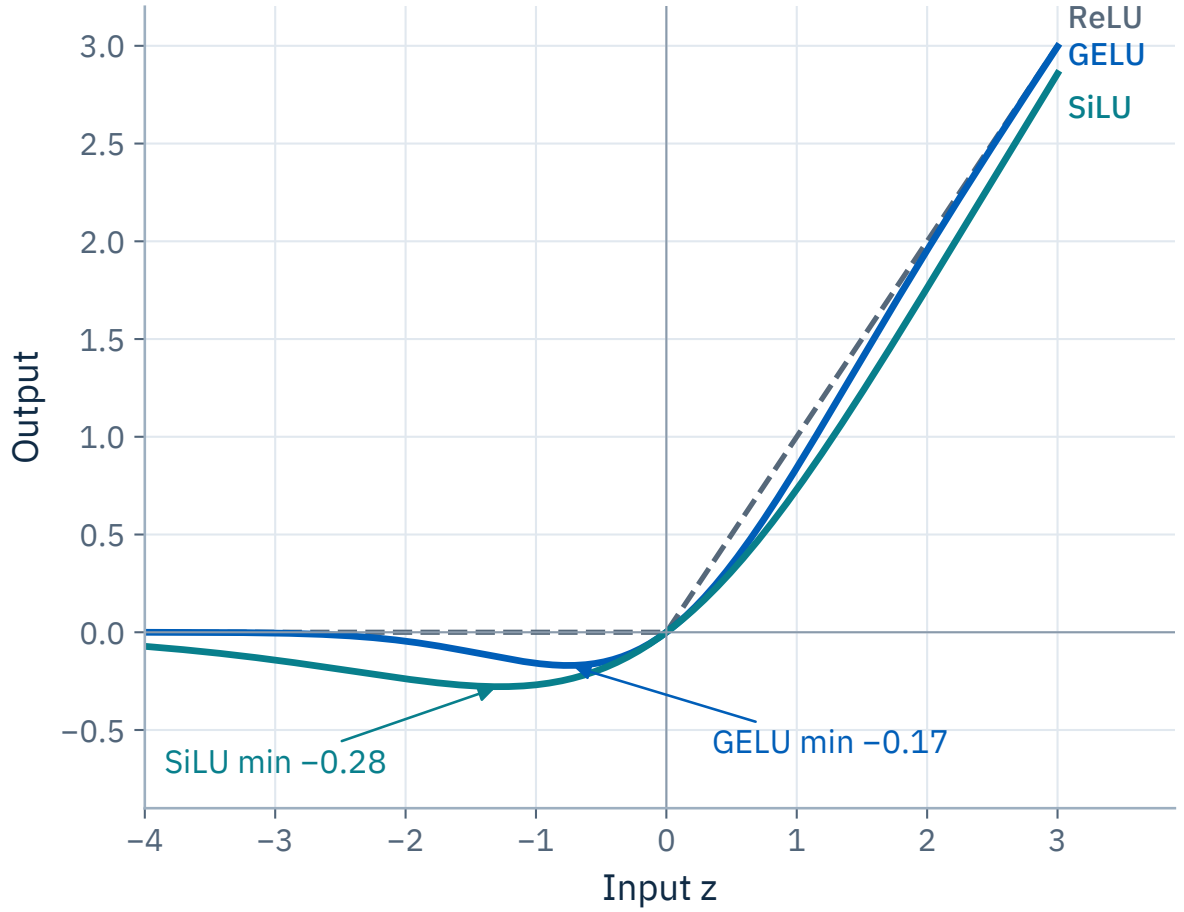


Analytical derivatives. ReLU's derivative at zero is undefined mathematically; software selects a convention.

GELU and SiLU provide smooth alternatives

$$\mathrm{GELU}(z) = z\,\Phi(z), \qquad \mathrm{SiLU}(z) = z\,\sigma(z)$$

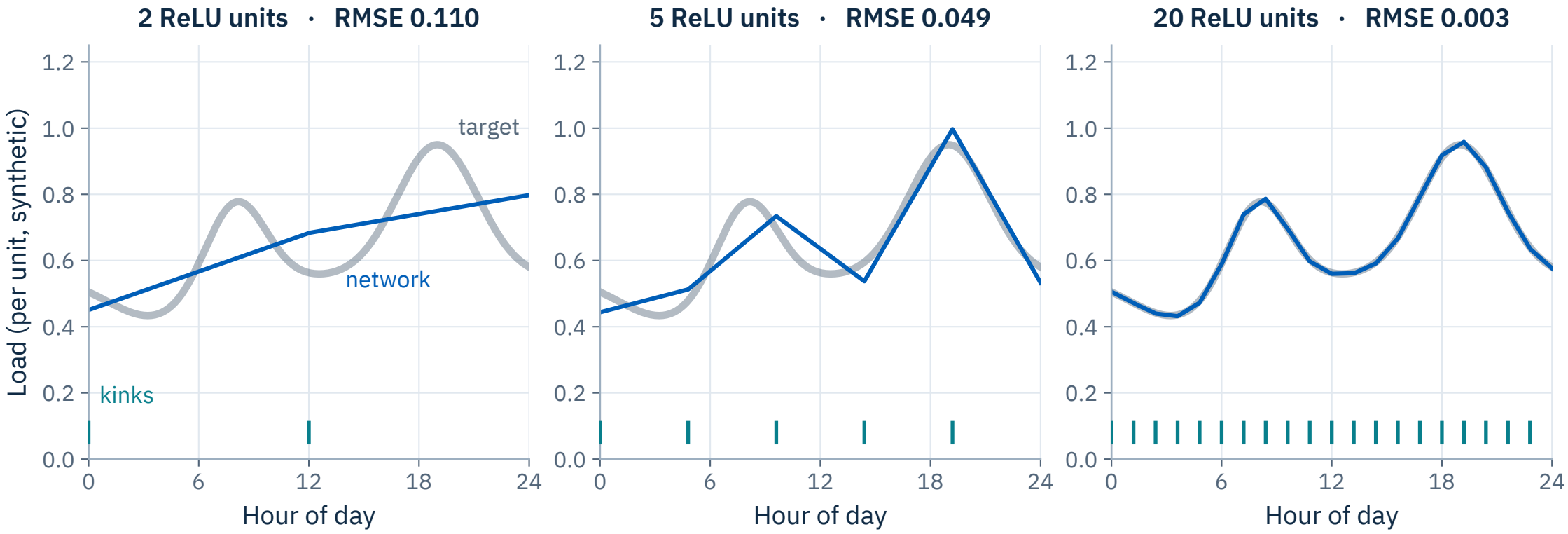
- GELU weights z by the standard-normal CDF Φ(z).
- SiLU, also called Swish, gates z with a sigmoid.
- Both are smooth and dip slightly below zero.
- Choose within a tested architecture, not in isolation.



Exact functions. Minima: GELU −0.17, SiLU −0.28.

Enough ReLU units can approximate a smooth curve

Each ReLU unit adds one kink; the output layer weights and sums them.



Computed: synthetic daily-load-shaped curve, not measured data. One hidden layer, evenly spaced kinks, output weights by least squares.

Universal approximation promises existence, not success

✓ What the theorem says

- One hidden layer with enough units
- Approximates any continuous function on a bounded domain
- Cybenko (1989); Hornik (1991)

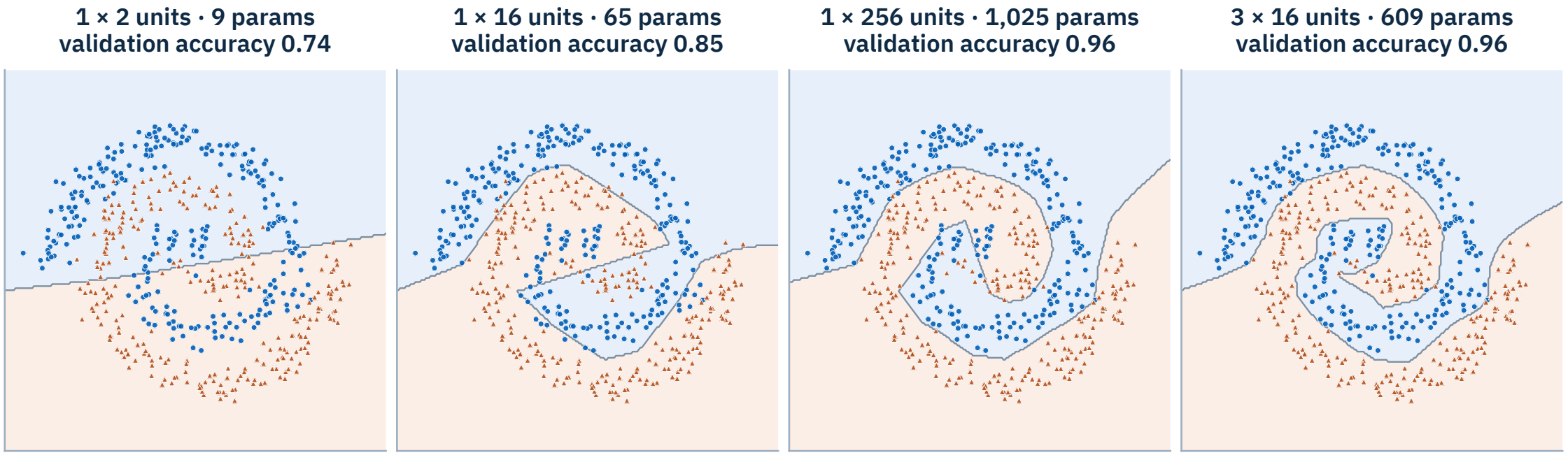
vs

✗ What it does not say

- How many units you will need
- Whether gradient descent will find the weights
- Whether the fit generalizes beyond the data

◆ Whether a network can represent your target is rarely the question; data, training, and validation are.

Width and depth both add capacity to the boundary



Computed: synthetic two spirals (seed 0, 70/30 split); scikit-learn MLPClassifier defaults (ReLU, Adam), random_state 0. Dots: training points; accuracy on the 30% validation points.

PRACTICE

3 min · Individual

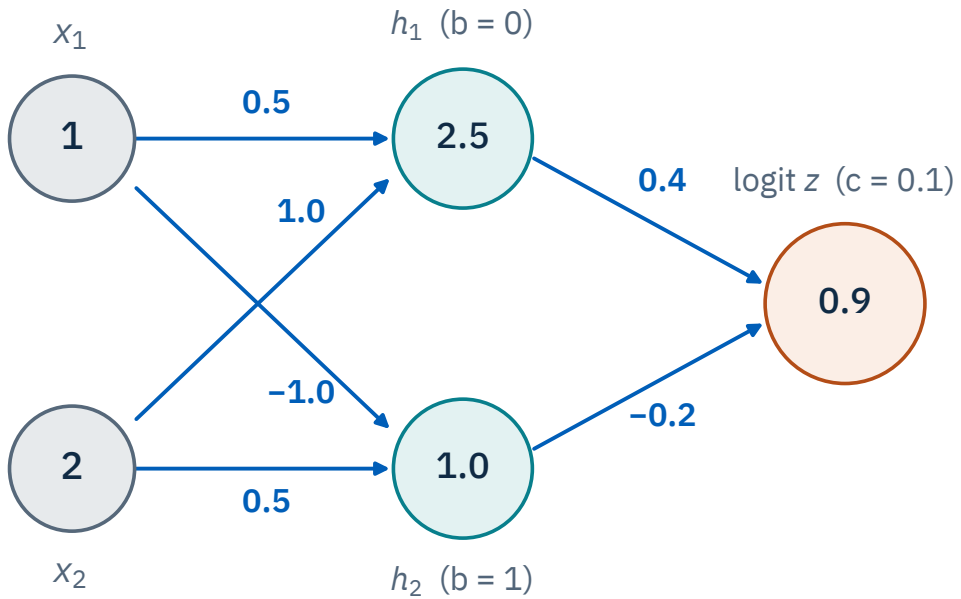
Dimensions determine parameter count

A dense network maps 6 inputs to 8 hidden units, then to 1 output.

1. Count the weights and biases in each layer.
2. State the output shape for a batch of 32 observations.
3. How does doubling the batch size change the parameter count?

A two-hidden-unit forward pass by hand

- $h_1 = \text{ReLU}(0.5 \cdot 1 + 1.0 \cdot 2 + 0) = 2.5$
- $h_2 = \text{ReLU}(-1.0 \cdot 1 + 0.5 \cdot 2 + 1) = 1.0$
- $z = 0.4 \cdot 2.5 - 0.2 \cdot 1.0 + 0.1 = 0.9$
- $p = \sigma(0.9) \approx 0.711$



$p = \sigma(0.9) = 0.711$ · ReLU keeps both hidden values

◆ Write these numbers down: Part 6 sends gradients back through this exact network.

Exact arithmetic. The weights are chosen for clear arithmetic, not fitted to data.

PRACTICE

4 min · Individual

Run the same network on a new input

Use the weights from the previous slide with $x = (3, 1)$.

1. Compute h_1, h_2, the logit z, and p.
2. Which hidden unit is inactive, and why?
3. Which weights receive zero gradient from this example?

PART 5 OF 7

05

Losses and gradient descent

The loss scores predictions; its gradient says how to improve.

- Output layers and matching losses
- Regression losses: MSE, MAE, and Huber
- Cross-entropy and softmax
- Gradient descent, batches, and the learning rate

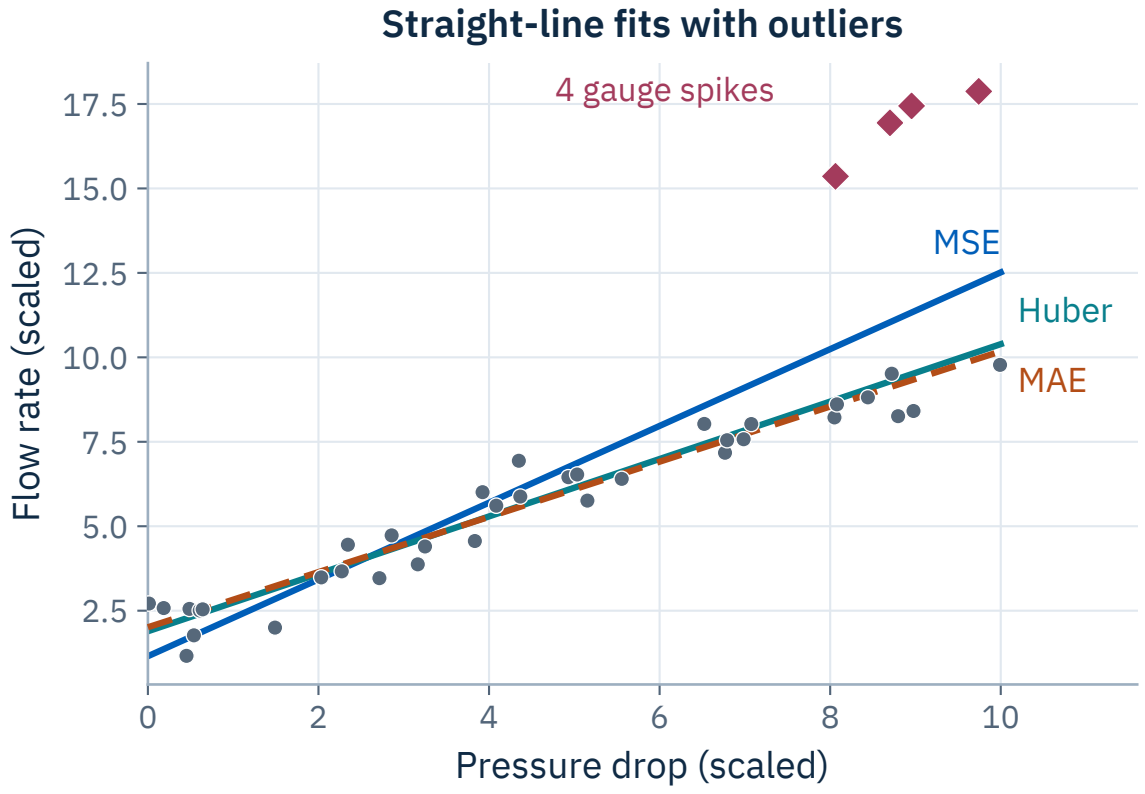
Output and loss must agree

Task	Model output	Training target and loss
Regression	Unconstrained numeric output	Numeric target, MSE or Huber
Binary classification	One raw logit per example	Float 0/1, BCEWithLogitsLoss
Multiclass classification	C raw logits per example	Integer class index, CrossEntropyLoss
Multilabel classification	One logit per independent label	Multiple 0/1 targets, binary losses

- Do not apply sigmoid or softmax before a loss that expects raw logits.

MSE, MAE, and Huber treat large errors differently





Left: exact loss functions. Right: synthetic data (seed 5), true slope 0.8, with 4 added spikes. Fitted slopes: MSE 1.14, MAE 0.82, Huber 0.85.

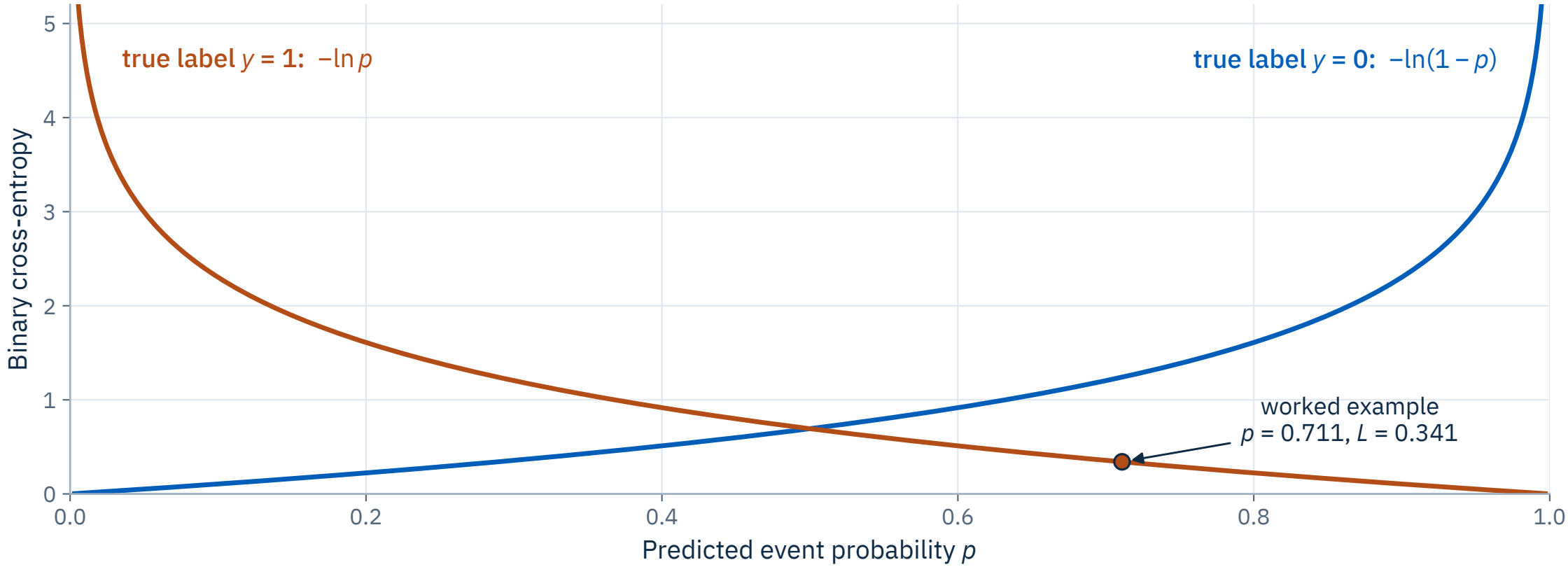
Binary cross-entropy rewards calibrated confidence

$$L = -[y \log p + (1 - y) \log(1 - p)]$$

- For y = 1, a low predicted probability incurs a high loss.
- For y = 0, a high predicted probability incurs a high loss.
- Stable implementations combine this with the sigmoid on raw logits.

◆ Worked example: y = 1 and p = 0.711 give L = −ln 0.711 ≈ 0.341.

Confident wrong answers cost the most



Exact binary cross-entropy for each possible label; the marked point is the worked example. Confident wrong probabilities incur much larger loss.

Softmax couples mutually exclusive classes

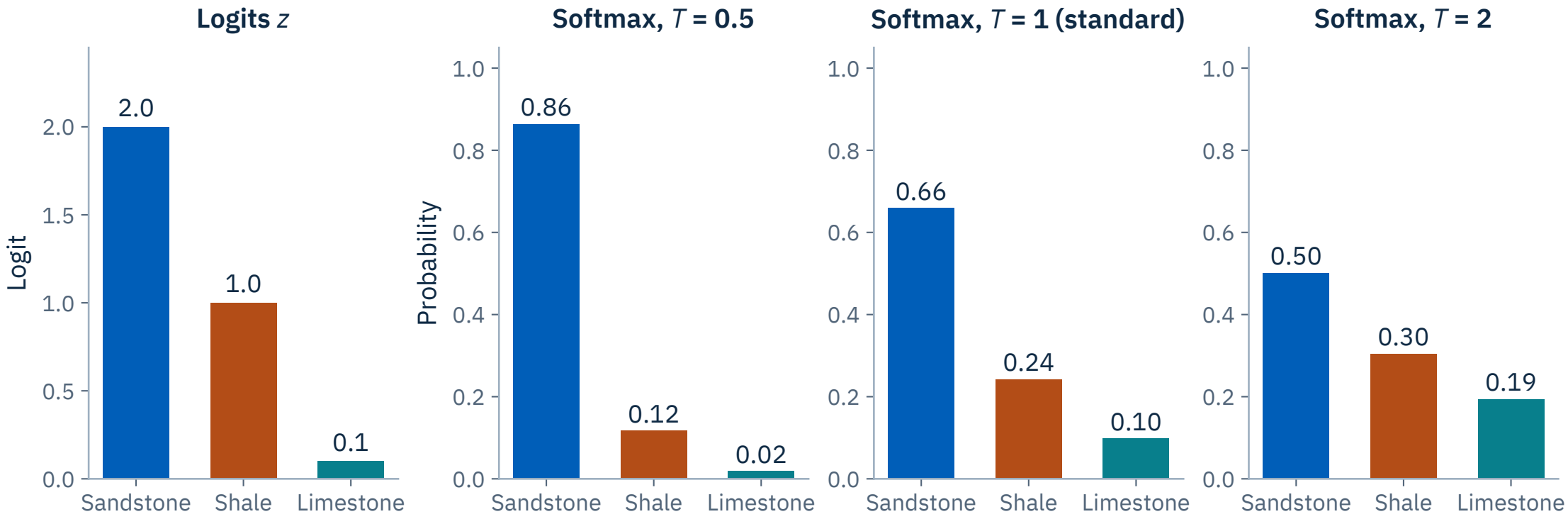
$$p_k = \frac{e^{z_k}}{\sum_j e^{z_j}}, \qquad L = -\log p_y$$

- The probabilities are positive and sum to one across classes.
- The target index *y* selects the correct class's probability.
- Adding the same constant to every logit changes nothing.
- `CrossEntropyLoss` applies the softmax internally, stably.

◆ Libraries subtract the largest logit before exponentiating; the probabilities are unchanged.

Temperature sharpens or flattens the softmax

Divide the logits by T before the softmax: $T < 1$ sharpens, $T > 1$ flattens, and the ranking never changes.



Exact softmax of illustrative facies logits (2.0, 1.0, 0.1) at three temperatures.

PRACTICE

4 min · Pairs

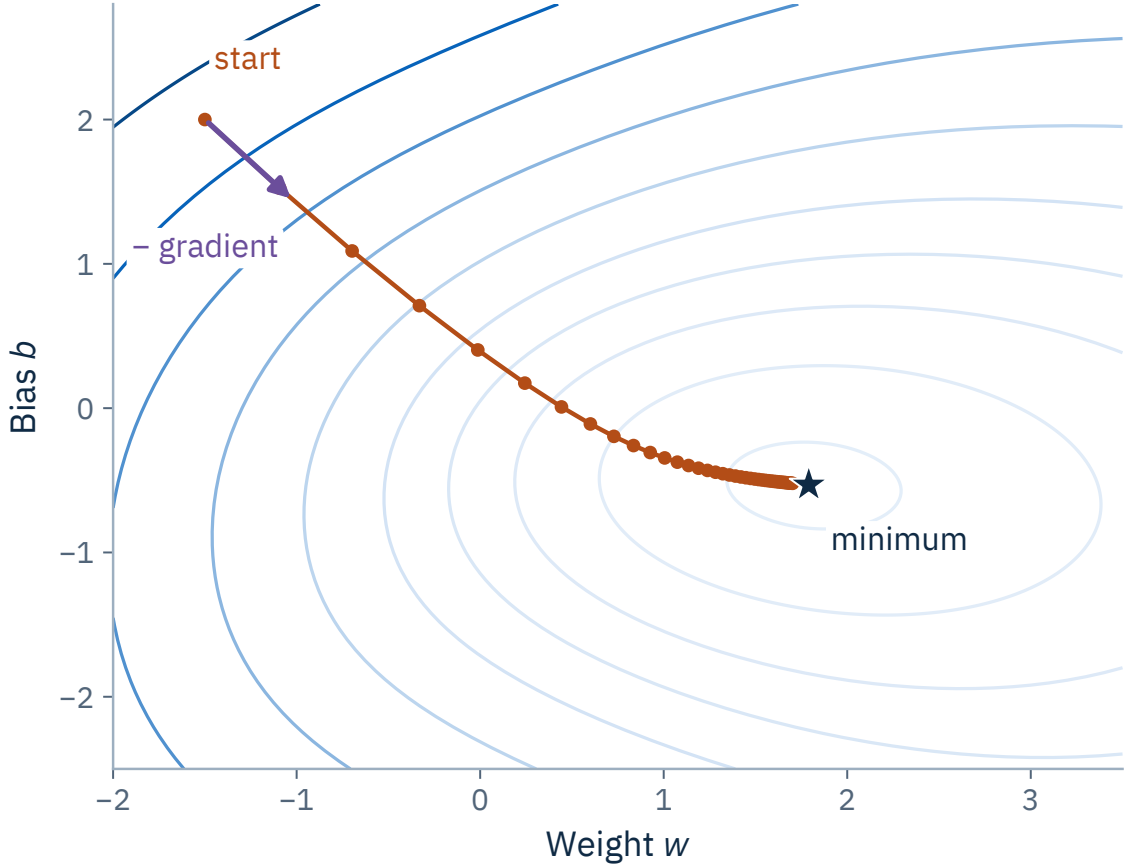
Compute a softmax and its loss

1. Logits (0, 0, 0): what are the probabilities, and the loss for any target?
2. Logits (2.0, 1.0, 0.1) with target Shale (index 1): what is the loss?
3. Add 5 to every logit: what changes?

Gradient descent steps against the gradient

$$w_{t+1} = w_t - \eta \nabla_w L(w_t)$$

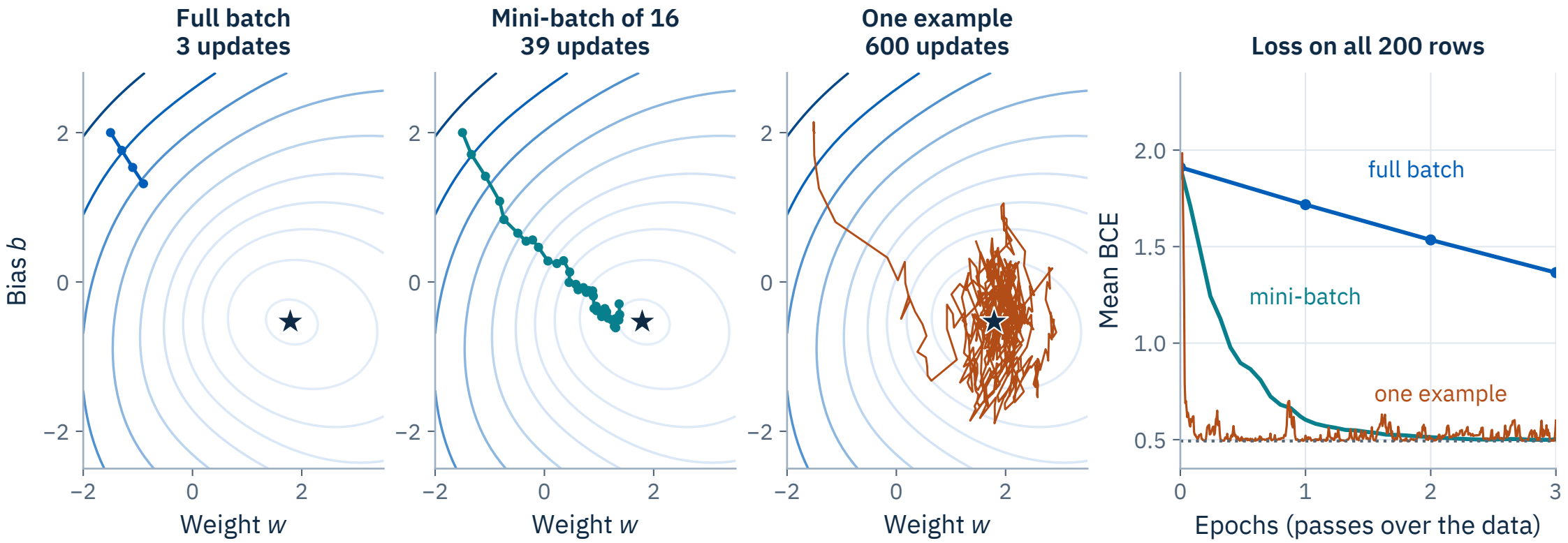
- The gradient points uphill; step the other way.
- The learning rate η sets the step size.
- Steps shrink as the surface flattens near the minimum.



Computed: loss surface of a one-feature logistic model on synthetic data (seed 7); 40 full-batch steps, learning rate 1.0.

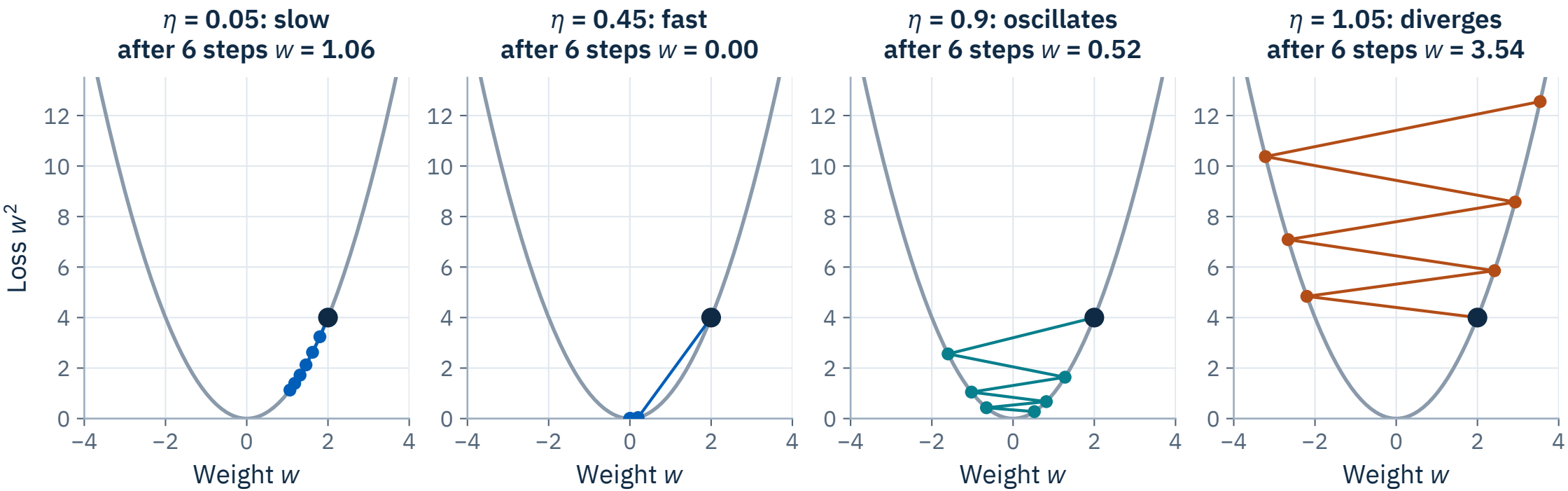
Batch size trades gradient noise for update count

Same data, start, learning rate, and 3 epochs: smaller batches make more, noisier updates.



Computed on the synthetic logistic problem: full batch (3 updates), mini-batches of 16 (39), single examples (600). Dotted line: minimum loss.

The learning rate decides whether descent converges



Exact gradient descent on L(w) = w² from w = 2; each step multiplies w by (1 − 2η). Schedules and adaptive optimizers follow on Day 2.

PRACTICE

4 min · Pairs

Match each output layer to its loss

For a batch of 64, give the output shape and the PyTorch loss for each task.

1. Daily gas rate of a well
2. Gas lock: yes or no
3. Facies: one of five classes
4. Five alarms that can fire together

PART 6 OF 7

06

Backpropagation

The chain rule, run in reverse, gives every gradient at once.

- The chain rule on a computational graph
- A full backward pass on the worked network
- Reverse mode versus finite differences
- Checking gradients numerically

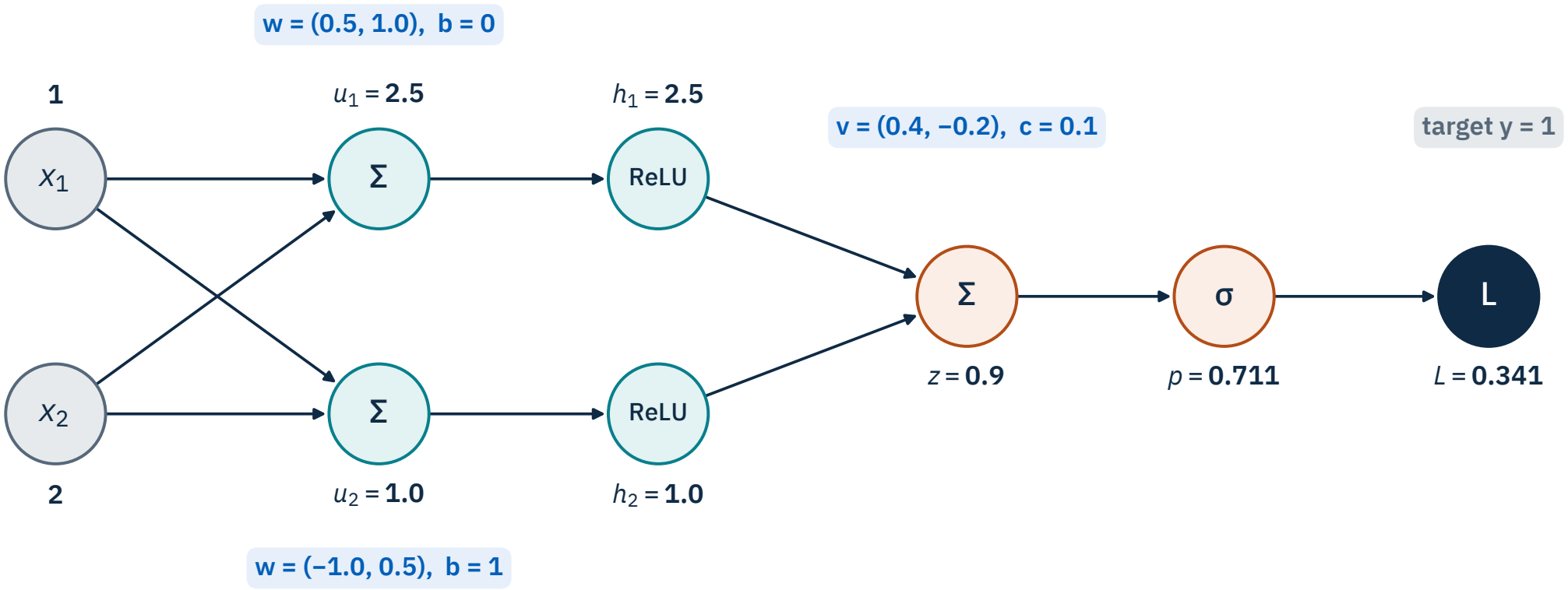
Backpropagation applies the chain rule

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial z}\,\frac{\partial z}{\partial w}$$

- Forward pass: compute and store every intermediate value.
- Backward pass: propagate sensitivities from the loss to the parameters.
- An optimizer then uses the gradients to update the parameters.

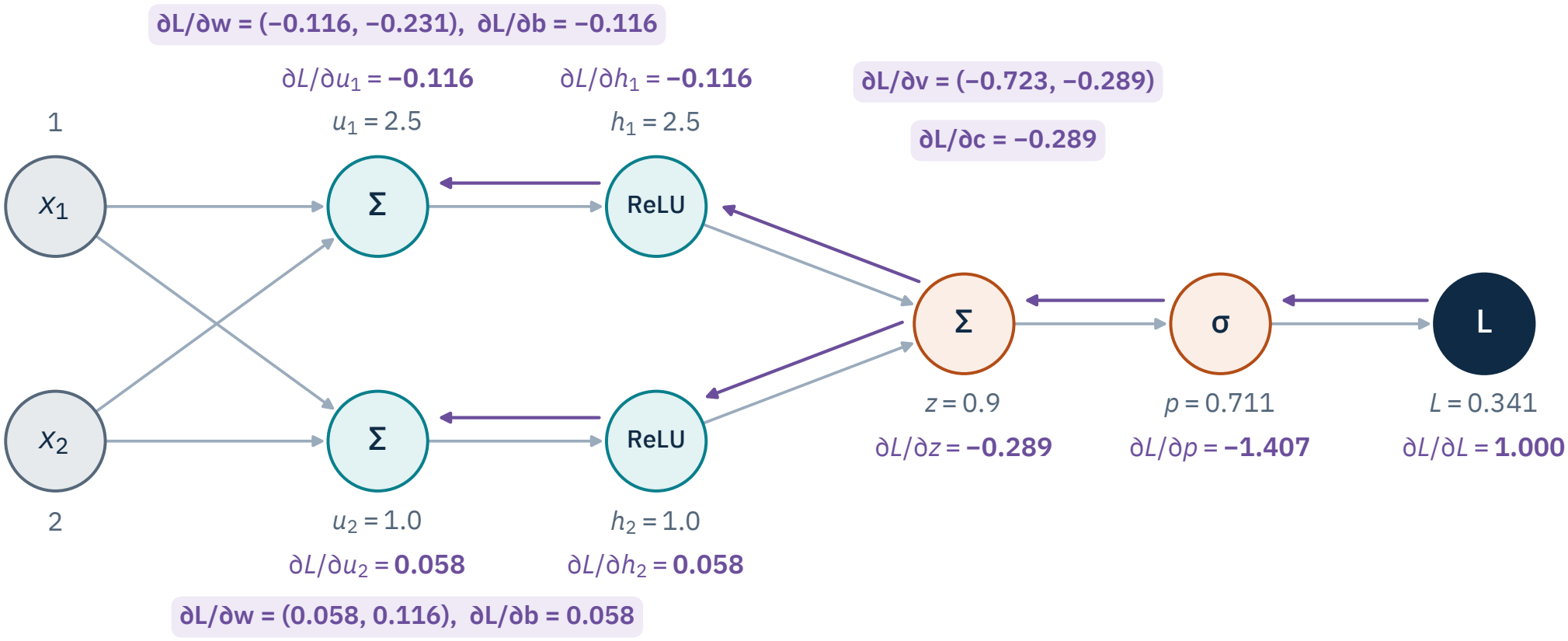
◆ For $z = v \cdot h + c$, the local derivative $\partial z / \partial v_1$ is simply h_1, a value stored by the forward pass.

A computational graph records the forward pass



Exact values for the worked network with x = (1, 2) and target y = 1. Σ: weighted sum; σ: sigmoid; L: binary cross-entropy.

Gradients flow backward through the same graph



Exact reverse-mode gradients. Each purple value is the upstream gradient times the node's local derivative.

Backpropagation is reverse-mode differentiation

✗ Finite differences

- Perturb one parameter, rerun the forward pass
- Two passes per parameter: millions of passes
- Approximate and sensitive to the step size

vs

✓ Reverse mode

- One forward pass and one backward pass
- All gradients for a small multiple of the forward cost
- Exact up to floating-point rounding

◆ PyTorch's autograd records the graph in the forward pass and runs reverse mode on `loss.backward()`.

One backward calculation updates one weight

$$\frac{\partial L}{\partial z} = p - y$$

- In the worked example, p = 0.711 and y = 1, so ∂L/∂z = −0.289.
- The output weight on h_1 has gradient −0.289 × 2.5 = −0.723.
- A learning rate of 0.1 moves that weight from 0.4 to about 0.472.

◆ Sigmoid with cross-entropy, like softmax with cross-entropy, gives "prediction minus target".

PRACTICE

5 min · Pairs

Push the gradient one layer further

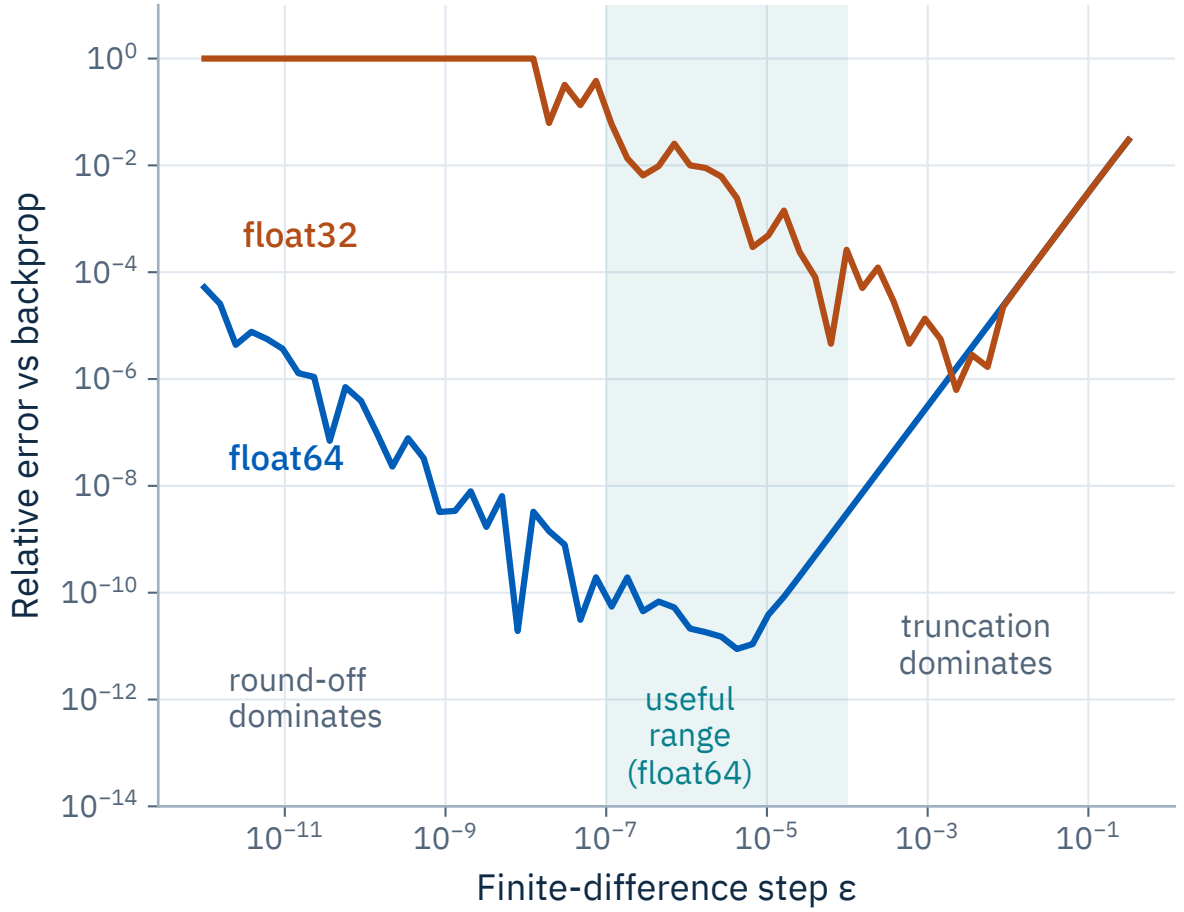
Worked network, $x = (1, 2)$, $y = 1$, and $p - y = -0.289$.

1. Find ∂L/∂w for the weight from x_2 to h_1 (value 1.0).
2. Update it with learning rate 0.1.
3. Repeat both steps for the weight from x_2 to h_2 (value 0.5).

A numerical gradient check catches silent errors

$$g_{\text{num}} = \frac{L(w+\epsilon) - L(w-\epsilon)}{2\epsilon}$$

- Compare with backpropagation using a relative error.
- Use float64 and ε around 10^{-5} to 10^{-6}.
- Too large ε: truncation error; too small: round-off.
- Avoid kinks, such as ReLU exactly at zero.



Computed for ∂L/∂v₁ of the worked network: relative error against the step ε.

A gradient check for the Lab 2 network

```python
def grad_check(loss_fn, params, grads, key, idx, eps=1e-5):
    old = params[key][idx]
    params[key][idx] = old + eps
    up = loss_fn(params)
    params[key][idx] = old - eps
    down = loss_fn(params)
    params[key][idx] = old                          # restore the weight
    num, ana = (up - down) / (2 * eps), grads["d" + key][idx]
    return abs(num - ana) / max(abs(num), abs(ana), 1e-12)

loss_fn = lambda p: binary_cross_entropy(y_train, forward(X_train, p)["a2"])
grads = backward(X_train, y_train, params, forward(X_train, params))
for key, idx in [("W1", (0, 0)), ("b1", (0, 3)), ("W2", (5, 0))]:
    print(key, idx, f"{grad_check(loss_fn, params, grads, key, idx):.1e}")
```

OUTPUT

```
W1 (0, 0) 3.8e-10
b1 (0, 3) 1.1e-08
W2 (5, 0) 2.5e-11
```

PART 7 OF 7

07

Training networks in code

Repeat forward, loss, backward, and step until validation says stop.

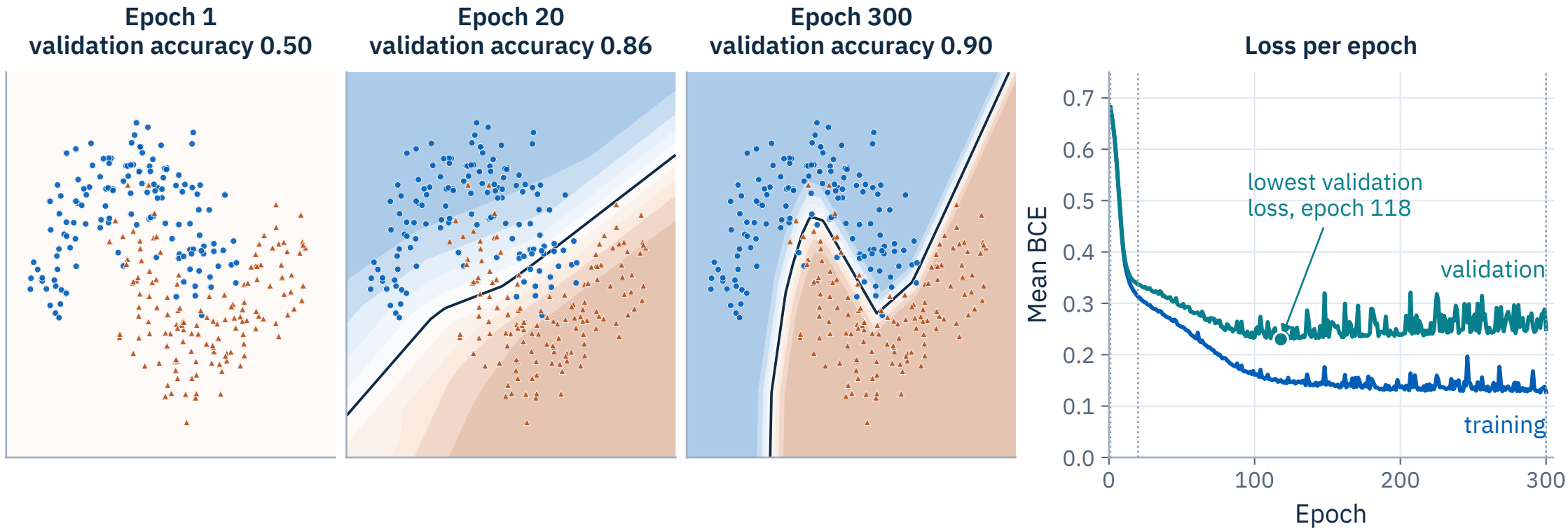
- The training loop and what each line does
- Watching the boundary and the losses evolve
- A first network in scikit-learn and in PyTorch
- Shape checks that catch first-day bugs

The training loop has four responsibilities

```
for epoch in range(n_epochs):
    model.train()
    for X_batch, y_batch in train_loader:
        optimizer.zero_grad()        # 1 clear
        logits = model(X_batch)      # 2 forward
        loss = criterion(logits, y_batch)
        loss.backward()              # 3 gradients
        optimizer.step()             # 4 update
    model.eval()
    with torch.no_grad():
        val_loss = criterion(model(X_val), y_val)
```

- Clear old gradients: PyTorch accumulates them.
- Forward pass and loss on one mini-batch.
- `backward()` fills every parameter's `.grad`.
- `step()` applies the update rule.
- Validate without building a graph.

The decision boundary takes shape during training



Computed: synthetic moons (seed 1); PyTorch 2–16–16–1 ReLU network, SGD with learning rate 0.1 and batch 32. Dotted lines: the three snapshot epochs. Validation loss is lowest at epoch 118.

Training loss is not the final score

- Monitor a separate validation set during development.
- Compare with the same baseline under the same split.
- Inspect learning curves and errors, not only the last epoch.
- Evaluate the chosen model once on the final test data.

◆ A model selected by looking at the test set has no unbiased score left.

A first network in scikit-learn

```
from sklearn.neural_network import MLPClassifier

mlp = make_pipeline(
    StandardScaler(),
    MLPClassifier(hidden_layer_sizes=(32,), alpha=1e-3,
                  max_iter=2000, random_state=0),
)
scores = cross_val_score(mlp, X_train, y_train, cv=cv)
print(f"CV accuracy {scores.mean():.3f} +/- {scores.std():.3f}")
```

OUTPUT

```
CV accuracy 0.843 +/- 0.052
```

Same folds as the logistic baseline (0.753 ± 0.061): compare fold by fold before celebrating.

A first network in PyTorch

```python
import torch
from torch import nn

class SonarNet(nn.Module):
    def __init__(self, n_in=60, n_hidden=32):
        super().__init__()
        self.layers = nn.Sequential(
            nn.Linear(n_in, n_hidden), nn.ReLU(),
            nn.Linear(n_hidden, 1))              # one raw logit

    def forward(self, x):                        # x: (B, 60) float32
        return self.layers(x)                    # (B, 1) logits

model = SonarNet()
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
print(sum(p.numel() for p in model.parameters()))
```

OUTPUT

```
1985
```

Autograd reproduces the gradients computed by hand

```python
import torch

x = torch.tensor([1.0, 2.0])
W1 = torch.tensor([[0.5, 1.0], [-1.0, 0.5]], requires_grad=True)
b1 = torch.tensor([0.0, 1.0], requires_grad=True)
v = torch.tensor([0.4, -0.2], requires_grad=True)
c = torch.tensor(0.1, requires_grad=True)
h = torch.relu(W1 @ x + b1)                    # (2.5, 1.0)
z = v @ h + c                                  # logit 0.9
loss = torch.nn.functional.binary_cross_entropy_with_logits(
    z, torch.tensor(1.0))
loss.backward()
print(round(loss.item(), 4), v.grad, W1.grad, sep="\n")
```

OUTPUT

```
0.3412
tensor([-0.7226, -0.2891])
tensor([[-0.1156, -0.2312],
        [ 0.0578,  0.1156]])
```

Shape checks turn silent bugs into loud ones

```
X_t = torch.as_tensor(X_train, dtype=torch.float32)   # (166, 60)
y_t = torch.as_tensor(y_train, dtype=torch.float32)   # (166,)
criterion = nn.BCEWithLogitsLoss()
logits = model(X_t)                                   # (166, 1)
assert X_t.shape[1] == 60 and torch.isfinite(X_t).all()
print(logits.shape, y_t.shape)
loss = criterion(logits, y_t)          # raises: the shapes differ
```

OUTPUT

```
torch.Size([166, 1]) torch.Size([166])
ValueError: Target size (torch.Size([166])) must be the same as
input size (torch.Size([166, 1]))
```

Fix with `y_t = y_t.reshape(-1, 1)`. `MSELoss` would only warn, then broadcast to (166, 166).

Choose scikit-learn for speed, PyTorch for control

Aspect	scikit-learn MLPClassifier	PyTorch
Architectures	Dense layers only	Any differentiable model
Training loop	Built into `fit`	You write it
Losses and outputs	Fixed by the estimator	You choose and check them
Hardware	CPU	CPU or GPU
Best use	Quick tabular baseline	Custom models, Days 2 to 5

◆ Lab 2 builds the same network a third way, by hand in NumPy, to see every step.

LABORATORY

45 min

Lab 2: a NumPy network you can explain

- ☐ Implement sigmoid, initialization, forward pass, BCE, and backward pass.
- ☐ Assert that every gradient has the shape of its parameter.
- ☐ Add the gradient check and report three relative errors.
- ☐ Train, plot the loss, and evaluate once on the test rows.
- ☐ Compare with the Lab 1 baseline; sections 12–13 are extensions.

NOTEBOOK

`exercises/02_nn_from_scratch/NN_From_Scratch_Student.ipynb`

EVIDENCE TO SUBMIT

- Shapes of W1, b1, W2, b2 and of their gradients
- Gradient-check relative errors in float64
- Loss curve, and train and test accuracy against the baseline

DISCUSSION

Lab 2 debrief: the loss fell, the test score is uncertain

Solution settings: 32 sigmoid units, learning rate 0.01, 1000 full-batch epochs, seed 42.

0.689 → 0.448

training loss, first to last epoch

0.843

training accuracy, 140 of 166

0.833

test accuracy, 35 of 42

2.4 points

the weight of one test echo

◆ Section 12 compares settings on validation rows; the test score belongs to the fixed settings.

DISCUSSION

Lab 2 debrief: what the small dataset cannot establish

Model ranking

- Test gaps are two echoes at most
- Rank models on the same CV folds
- Report spread, not only means

Chosen settings

- Fixed before the single test
- Compared on validation rows only
- A re-tuned model needs a fresh test

Deployment claims

- One lab dataset, no acquisition metadata
- No evidence about new sites or sensors
- Treat results as a teaching benchmark

Key takeaways

- ✓ A network composes weighted sums and nonlinearities; shapes are its contract.
- ✓ One line cannot separate XOR; hidden units re-map inputs until one can.
- ✓ Choose the output activation, target encoding, and loss together.
- ✓ Backpropagation is the chain rule in reverse; check it numerically once.
- ✓ Training loss falls by design; only held-out data show generalization.
- ✓ A network must beat a baseline on the same split to earn its place.

LOOKING AHEAD

Tomorrow: training neural networks reliably

Diagnose

- Initial loss, one-batch fit, learning curves

Initialize and normalize

- Keep signals and gradients at a trainable scale

A loop you can trust

- Checkpoints, seeds, logs, and the Keras equivalent

Optimize

- Momentum, AdamW, schedules, batch size

Regularize

- Weight decay, dropout, early stopping, augmentation

Autoencoders

- Reconstruction error as an anomaly score

◆ Every Day 2 tool plugs into today's loop: forward, loss, backward, step.

PRACTICE

5 min | Individual

Exit check

1. Why do dense layers need nonlinear activations?
2. Why must the raw-logit loss and the output activation agree?
3. Which shape error can pass silently, and how do you catch it?
4. How can a gradient check reveal a silent implementation error?

References and further study

- Goodfellow, Bengio, Courville (2016). *Deep Learning*. MIT Press. Chapters 6 and 8.
- Rumelhart, Hinton, Williams (1986). Learning representations by back-propagating errors. *Nature* 323.
- Rosenblatt (1958). The perceptron. *Psychological Review* 65.
- Minsky, Papert (1969). *Perceptrons*. MIT Press.
- Cybenko (1989); Hornik (1991): universal approximation theorems.
- PyTorch tutorials: tensors, autograd, and the training loop. scikit-learn User Guide: neural network models.
- UCI Machine Learning Repository: Connectionist Bench (Sonar, Mines vs. Rocks).